

CSCI-4208: Developing Advanced Web Applications

Week 11: Lectures 21-23

MongoDB Basics

MongoDB Basics

1. What is MongoDB?
2. Importing, Exporting, & Querying Data
3. Creating & Manipulating Data
4. Advanced CRUD Operations
5. Indexing & Aggregation Pipeline
6. Next Steps

1. What is MongoDB?

Part 1

What is MongoDB?

1. What is MongoDB?

- **What is the MongoDB Database?**
- **What is a Document in MongoDB?**
- **What is MongoDB Atlas?**
- **What is Mongo Shell?**
- **Mongo Shell - Basic Commands**

1. What is the MongoDB Database?

What is the
MongoDB Database?

1. What is the MongoDB Database? - Collections

Collection - an organized store of documents in MongoDB usually with common fields between documents. There can be many collections per database and many documents per collection.

1. What is the MongoDB Database? - NoSQL

MongoDB is a database, meaning a structured way to store and access data.

More specifically, it is a NoSQL database. NoSQL is a very generic term used to describe any data store that doesn't use a legacy approach of related tables of data. This means that you're storing your data in an organized way, but not in rows and columns

An example of NoSQL databases can be anything from a library card catalog, to a more sophisticated data store like MongoDB. Since the range of what NoSQL means is so wide, we must narrow our definition further.

MongoDB is a NoSQL document database. This means that data in MongoDB is stored as documents. These documents are in turn stored in what we call collections of documents.

1. What is the MongoDB Database? (Quiz 1)

Why is MongoDB a NoSQL database?

- Because it uses a structured way to store and access data
- Because it does not utilize tables, rows, and columns to organize data.

1. What is a Document in MongoDB?

What is a Document
in MongoDB?

1. What is a Document in MongoDB? - Terms

Document

a way to organize and store data as a set of field-value pairs.

Field

a unique identifier for a datapoint.

Value

data related to a given identifier.

Collection

an organized store of documents in MongoDB, usually with common fields between documents. There can be many collections per database and many documents per collection.

1. What is the MongoDB Database? (Quiz 1)

MongoDB database:

- The MongoDB database is an organized way to store and access data
- MongoDB's is a NoSQL database that uses documents to store data in an organized way.

Why is MongoDB a NoSQL?

- MongoDB does **NOT** organize documents in rows and columns
- MongoDB database does **NOT** use tables of related data

1. What is a Document? (Quiz 2)

In MongoDB how does a document relate to a collection?

Collections consist of one or many documents

1. What is a Document? (Quiz 3)

In a MongoDB Document what is the role of fields and values?

- A field is a unique identifier for a specific datapoint
- Each field has a value associated with it.

1. What is MongoDB Atlas?

What is MongoDB
Atlas?

1. What is MongoDB Atlas

MongoDB Atlas is a fully-managed cloud database developed by the same people that build MongoDB. Atlas handles all the complexity of deploying, managing, your database deployments.

- Use MongoDB database for data storage & retrieval.
- MongoDB Atlas is a hosting service for your MongoDB database instance.

1. What is MongoDB Atlas

Cloud-based Database Architecture (MongoDB Atlas):

Replica Set

a few connected machines that store the same data to ensure that if something happens to one of the machines the data will remain intact. Comes from the word replicate - to copy something.

Instance

a single machine locally or in the cloud, running a certain software, in this case it is the MongoDB database.

Cluster

group of servers that store your data.

1. What is MongoDB Atlas

MongoDB Atlas has a free tier (Shared Cluster):

Shared Cluster

A secure and highly available replica set designed for learning, building, prototypes, and early development.

Backup

Daily snapshot backup is included for M2 (2 GB storage) and M5 (5 GB) deployments.

Storage

512 MB to 5 GB

RAM

Shared

Starting at

Free

Start Free

1. What is Atlas? (Quiz 1)

How is MongoDB Atlas related to MongoDB the Database?

- Atlas has many tools and services within it that are built specifically for the MongoDB Database.
- They are both MongoDB products.

1. What is Mongo Shell?

What is
Mongo Shell?

1. What is Mongo Shell?

The mongo shell is an interactive JavaScript interface to MongoDB. You can use the mongo shell to query and update data as well as perform administrative operations.

Note: Mongo Shell may be used with any Mongo deployment, not just from Atlas.

Download & Install:

<https://www.mongodb.com/try/download/shell>

1. What is Mongo Shell?

Connect to an Atlas cluster with Mongo Shell

```
mongo "mongodb+srv://<username>:<password>@<cluster>.mongodb.net/admin"
```

Mongo Shell after connecting to a cluster using the above command

```
Implicit session: session { "id" : UUID("cb0a69a9-4208-4c28-8ed9-060cd8ec3a48") }  
MongoDB server version: 4.4.4  
MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY> █
```

From this shell you can interact with your MongoDB database.

1. Mongo Shell - Basic Commands

Mongo Shell - Basic Commands

1. Mongo Shell - Basic Commands

> help

```
db.help()          help on db methods
db.mycoll.help()   help on collection methods
sh.help()          sharding helpers
rs.help()          replica set helpers
help admin         administrative help
help connect       connecting to a db help
help keys          key shortcuts
help misc          misc things to know
help mr            mapreduce

show dbs           show database names
show collections   show collections in current database
show users         show users in current database
show profile       show most recent system.profile entries with time >= 1ms
show logs          show the accessible logger names
show log [name]    prints out the last segment of log in memory, 'global' is default
use <db_name>      set current database
db.mycoll.find()   list objects in collection mycoll
db.mycoll.find( { a : 1 } ) list objects in mycoll where a == 1
it                result of the last line evaluated; use to further iterate
DBQuery.shellBatchSize = x set default number of items to display on shell
exit              quit the mongo shell
```

1. Mongo Shell - Basic Commands

> show dbs

```
[MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY> show dbs
sample_airbnb
sample_analytics
sample_geospatial
sample_mflix
sample_restaurants
sample_supplies
sample_training
sample_weatherdata
admin
local
```

This command shows all the databases in this connected MongoDB instance

1. Mongo Shell - Basic Commands

> use <db name>

```
[MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY> use sample_training  
switched to db sample_training
```

This command sets the current database referenced as db

> db

```
[MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY> db  
sample_training
```

db is an alias for the current database

1. Mongo Shell - Basic Commands

> show collections

```
[MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY> show collections  
companies  
grades  
inspections  
posts  
routes  
trips  
zips
```

This command shows all the collections in the database

2. Importing, Exporting, & Querying Documents

Part 2

Importing, Exporting, & Querying Documents

2. Importing, Exporting, & Querying Data

- **How does MongoDB store data**
 - What is JSON
 - What is BSON
- **Importing & Exporting Data**
- **Atlas - Data Explorer**
 - find
- **MQL - find command**
 - MQL - find
 - MQL - findOne
 - MQL - count
 - MQL - pretty

2. How does MongoDB store data?

How does MongoDB
store data?

2. How does MongoDB store data?

MongoDB uses documents to store data. How are documents represented as memory and what is the correct syntax for these objects that we call documents?

When you view or update documents in Mongo Shell, you're working in JSON.

JSON: JavaScript Object Notation

JSON format:

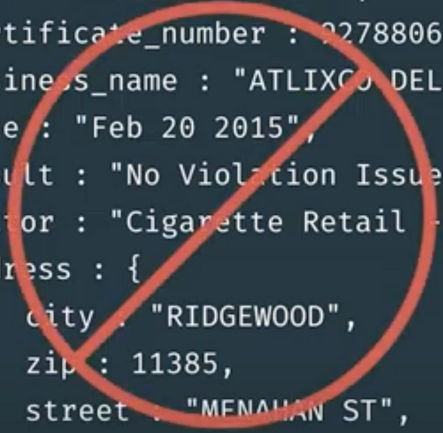
- Start and end with curly braces `{ }`
- Separate each **key** and **value** with a colon `:`
- Separate each **key:value** pair with a comma `,`
- All keys are surrounded by quotation marks `"`
- All string values must use quotation marks `"`
- In MongoDB **"keys"** are called **"fields"**

2. How does MongoDB store data?

JSON Example:

```
{
  "_id" : "10021-2015-ENFO",
  "certificate_number" : 9278806,
  "business_name" : "ATLIXCO DELI",
  "date" : "Feb 20 2015",
  "result" : "No Violation Issued",
  "sector" : "Cigarette Retail - 127",
  "address" : {
    "city" : "RIDGEWOOD",
    "zip" : 11385,
    "street" : "MENAHAN ST",
    "number" : 1712
  }
}
```

```
[
  {
    "_id" : "10021-2015-ENFO",
    "certificate_number" : 9278806,
    "business_name" : "ATLIXCO DELI",
    "date" : "Feb 20 2015",
    "result" : "No Violation Issued",
    "sector" : "Cigarette Retail - 127",
    "address" : {
      "city" : "RIDGEWOOD",
      "zip" : 11385,
      "street" : "MENAHAN ST",
      "number" : 1712
    }
  }
]
```



2. How does MongoDB store data?

JSON Example:

```
{
  "_id" : "10021-2015-ENFO",
  "certificate_number" : 9278806,
  "business_name" : "ATLIXCO DELI",
  "date" : "Feb 20 2015",
  "result" : "No Violation Issued",
  "sector" : "Cigarette Retail - 127",
  "address" : {
    "city" : "RIDGEWOOD",
    "zip" : 11385,
    "street" : "MENAHAN ST",
    "number" : 1712
  }
}
```



A field that contains a document (object) as a value.

2. How does MongoDB store data?

Pros of JSON	Cons of JSON
Friendly	Text-based/slow parsing
Readable	Space-consuming
Familiar	Limited data types

**MongoDB address these drawbacks with BSON*

2. How does MongoDB store data?

BSON: Binary JSON

```
_id[0a2↵>E0<00  
saleDate"0uHLitems00mnameprinter  
papertags%0office1□stationaryprice0  
<0quantity1rnamenotepadtags00office  
1writing2schoolprice0  
<0quantity20namepenstagsB0writing1o  
ffice2school3□stationaryprice0<0qua  
ntity3pname  
backpacktags-0school1travel2kidspri  
ce[<0quantity4rnamenotepadtags00off  
ice1writing2schoolprice7<0quantity5  
xname  
envelopestags40  
stationary1office2generalprice0<0qu  
antity6xname
```

The JSON data is stored into memory with binary-encoded optimizations.

Bridges between binary representation & JSON format.

Optimized for:

- Speed
- Space
- Flexibility

High Performance

General-purpose focus

2. How does MongoDB store data?

JSON	BSON
Encoding UTF-8 String	Encoding Binary
Data Support String, Boolean, Number, Array	Data Support String, Boolean, Number (Integer, Long, Float, ...), Array, Date, Raw Binary
Readability Human and Machine	Readability Machine only

2. How does MongoDB store data?

BSON in Memory / JSON in Model

- MongoDB stores data in BSON format internally
- But you may view/update the data as if it were modeled as JSON.

2. What is JSON? (Quiz 1)

Which of the following documents is correct JSON?

```
{ "name" : "Devi", "age":  
19, "major": "Computer  
Science" }
```

```
[ "name" : "Devi", "age":  
19, "major": "Computer  
Science" ]
```

```
{ name : "Devi", age: 19,  
major: "Computer Science" }
```

2. What is JSON?

Which of the following documents is correct JSON?

```
{ "name" : "Devi", "age":  
19, "major": "Computer  
Science" }
```



```
[ "name" : "Devi", "age":  
19, "major": "Computer  
Science" ]
```



```
{ name : "Devi", age: 19,  
major: "Computer Science" }
```



2. JSON vs. BSON (Quiz 2)

- MongoDB stores data in _____ and you can then view it in _____
- _____ is faster to parse and lighter to store than _____
- _____ supports fewer data types than _____

2. JSON vs. BSON (Quiz 2)

- MongoDB stores data in **BSON** and you can then view it in **JSON**
- **BSON** is faster to parse and lighter to store than **JSON**
- **JSON** supports fewer data types than **BSON**

2. Importing & Exporting Data

Importing & Exporting Data

2. Importing & Exporting Data

Atlas allows for data to import & export from a cluster to different systems. Data may be imported or exported in either JSON or BSON formats.

JSON	BSON
mongoimport	mongorestore
mongoexport	mongodump

2. Importing & Exporting Data

Exports data in BSON

```
mongodump --uri "<Atlas Cluster URI>"
```

Exports data in JSON

```
mongoexport --uri "<Atlas Cluster URI>" --collection=<collection name> --out=<filename.json>
```

Imports data in BSON

```
mongoimport --uri "<Atlas Cluster URI>" --drop dump
```

Imports data in JSON

```
mongoimport --uri "<Atlas Cluster URI>" --drop=<filename.json>
```

2. Importing & Exporting Data (Quiz 1)

Problem:

Which of the following commands will add a collection that is stored in `animals.json` to an Atlas cluster?

- A. `mongoexport`
- B. `mongorestore`
- C. `mongodump`
- D. `mongoimport`

2. Importing & Exporting Data (Quiz 1)

Problem:

Which of the following commands will add a collection that is stored in animals.json to an Atlas cluster?

- A. mongoexport
- B. mongorestore
- C. mongodump
- D. mongoimport

2. Atlas - Data Explorer

Atlas - Data Explorer

2. Atlas - Data Explorer: find

There are many ways to query collections in MongoDB. One is directly from the Atlas using Data Explorer (GUI-based).

The screenshot shows the MongoDB Atlas Data Explorer interface. On the left is a sidebar with a list of collections: sample_analytics, sample_geospatial, sample_mflix, sample_restaurants, sample_supplies, sample_training (highlighted in green), companies, disaster, grades, inspections, posts, routes, trips, zips, and sample_weatherdata. An orange arrow labeled 'Collections' points to this list. At the top, there are tabs for 'Find', 'Indexes', 'Schema Anti-Patterns', and 'Aggregation'. An orange arrow labeled 'Find view' points to the 'Find' tab. Below the tabs is a 'FILTER' input field containing the JSON query `{ "filter": "example" }`. An orange arrow labeled 'Query form formatted in JSON' points to this field. Below the filter is a section titled 'QUERY RESULTS 1-20 OF MANY'. It displays two document entries. The first entry is expanded, showing its fields: `_id` (ObjectId), `city` (HUEYTOWN), `zip` (35023), `loc` (Object), `pop` (39677), and `state` (AL). An orange arrow labeled 'Document (entry)' points to this first document. The second entry is partially visible below it, showing `_id` (ObjectId), `city` (CHELSEA), `zip` (35043), and `loc` (Object).

Find view

Collections

Query form formatted in JSON

Document (entry)

2. Atlas - Data Explorer: find

Example 1 (Query):

Suppose you wanted to query the MongoDB collection for all documents where the state is New York

```
{"state": "NY"}
```

Example 2 (Query):

Suppose you wanted to query the MongoDB collection for all documents where the state is New York and the city is Albany:

```
{"state": "NY", "city": "ALBANY"}
```


2. MQL - find command

MQL -
find command

2. MongoDB: mongo shell

Now let's query data from the MongoDB database from a remote environment. These examples will connect using the mongo shell.

But these same methods work generally in any remote environment

The mongo shell

- allows you to interact with your MongoDB instance without using a GUI
- It is a fully functioning JavaScript Interpreter

2. MongoDB: find

find() returns a cursor with documents that match the find query

Syntax:

```
db.<collection name>.find({ "key": "value" })
```

Example 1: *(queries the zip collection for all documents containing state field with value "NY")*

```
db.zips.find({ "state": "NY" })
```

Example 2: *(queries zip collection for all documents where state is "NY" & city is "ALBANY")*

```
db.zips.find({ "state": "NY", "city": "ALBANY" })
```

2. MongoDB: it

- **it** a mongo command that iterates through a cursor.
- **cursor** - A pointer to a results set of query
- **pointer** - A direct address of the memory location

Example:

```
MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY> db.zips.find({"state":"NY"})
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f89"), "city" : "FISHERS ISLAND", "zip" : "06390", "loc" : { "y" : 41.263934, "x" : 72.017834 }, "pop" : 329, "state" : "NY" }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f8a"), "city" : "NEW YORK", "zip" : "10001", "loc" : { "y" : 40.74838, "x" : 73.996705 }, "pop" : 18913, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f8b"), "city" : "NEW YORK", "zip" : "10003", "loc" : { "y" : 40.731253, "x" : 73.989223 }, "pop" : 51224, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f8c"), "city" : "GOVERNORS ISLAND", "zip" : "10004", "loc" : { "y" : 40.693604, "x" : 74.019025 }, "pop" : 3593, "state" : "NY" }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f8d"), "city" : "NEW YORK", "zip" : "10005", "loc" : { "y" : 40.705649, "x" : 74.008344 }, "pop" : 202, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f8f"), "city" : "NEW YORK", "zip" : "10006", "loc" : { "y" : 40.708451, "x" : 74.013474 }, "pop" : 119, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f90"), "city" : "NEW YORK", "zip" : "10009", "loc" : { "y" : 40.726188, "x" : 73.979591 }, "pop" : 57426, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f92"), "city" : "NEW YORK", "zip" : "10010", "loc" : { "y" : 40.737476, "x" : 73.981328 }, "pop" : 24907, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f93"), "city" : "NEW YORK", "zip" : "10002", "loc" : { "y" : 40.715231, "x" : 73.987681 }, "pop" : 84143, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f94"), "city" : "NEW YORK", "zip" : "10012", "loc" : { "y" : 40.72553, "x" : 73.998284 }, "pop" : 26365, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f95"), "city" : "NEW YORK", "zip" : "10011", "loc" : { "y" : 40.740225, "x" : 73.99963 }, "pop" : 46560, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f96"), "city" : "NEW YORK", "zip" : "10007", "loc" : { "y" : 40.713905, "x" : 74.007022 }, "pop" : 3374, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f97"), "city" : "NEW YORK", "zip" : "10013", "loc" : { "y" : 40.718511, "x" : 74.002529 }, "pop" : 21860, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f98"), "city" : "NEW YORK", "zip" : "10014", "loc" : { "y" : 40.73393, "x" : 74.005421 }, "pop" : 31147, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f9a"), "city" : "NEW YORK", "zip" : "10017", "loc" : { "y" : 40.75172, "x" : 73.970661 }, "pop" : 12465, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f9b"), "city" : "NEW YORK", "zip" : "10018", "loc" : { "y" : 40.754713, "x" : 73.992503 }, "pop" : 4834, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f9c"), "city" : "NEW YORK", "zip" : "10019", "loc" : { "y" : 40.765069, "x" : 73.985834 }, "pop" : 36602, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72f9f"), "city" : "NEW YORK", "zip" : "10020", "loc" : { "y" : 40.759729, "x" : 73.982347 }, "pop" : 393, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72fa0"), "city" : "NEW YORK", "zip" : "10021", "loc" : { "y" : 40.768476, "x" : 73.958805 }, "pop" : 106564, "state" : "NY", "capital?" : false }
{ "_id" : ObjectId("5c8eccc1caa187d17ca72fa1"), "city" : "NEW YORK", "zip" : "10016", "loc" : { "y" : 40.744281, "x" : 73.978134 }, "pop" : 51561, "state" : "NY", "capital?" : false }

Type 'it' for more
MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY>
```

2. MongoDB: findOne

findOne method returns the first document to meet the query parameters

Syntax:

```
db.collection.findOne({ "key":"value" })
```

Example 1: *(empty parameter returns first document from the collection)*

```
db.zips.findOne({})
```

This method may be useful to inspect the keys of a document.

2. MongoDB: count

The find method may return many documents. You may chain a **count() method** after it. Returns the number of documents that match a given query.

Syntax:

```
db.<collection name>.find(<query>).count()
```

Example 1: *(get the count for the number of documents that contain the state "NY")*

```
db.zips.find({"state": "NY" }).count()
```

```
MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY> db.zips.find({"state":"NY"}).count()  
1596  
MongoDB Enterprise atlas-t6qh3x-shard-0:PRIMARY> db.zips.find({"state":"NY", "city":"ALBANY"}).count()  
7
```

2. MongoDB: pretty

The pretty method returns the documents that match the given query formatted for easy reading

Syntax:

```
db.<collection name>.find(<query>).pretty()
```

Example 1: *(prints the results of the query to be pretty JSON-formatted)*

```
db.zips.find({"state": "NY" }).pretty()
```

3. Creating & Managing Documents

Part 3

Creating & Managing Documents

3. Creating & Manipulating Documents

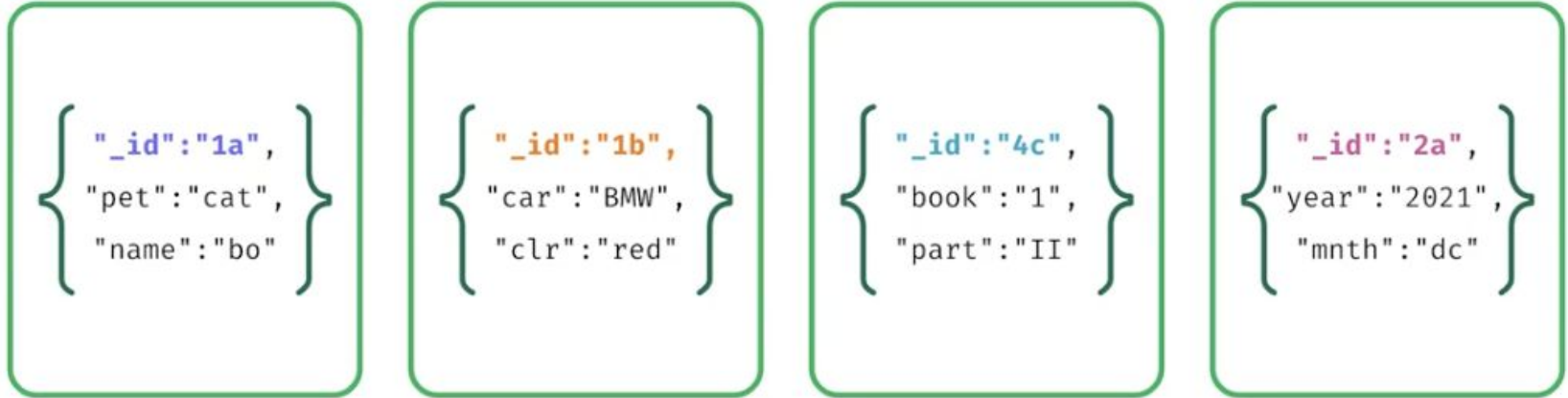
- **Create - ObjectID**
- **Create - MQL: insert**
 - **insert()**
 - **errors**
- **Create - Option: ordered**
- **Update**
- **Delete**

3. Create - ObjectID

Create -
ObjectID

3. Inserting New Documents - ObjectId

All newly created documents is populated with an `_id` field used as its unique key.



3. Inserting New Documents - ObjectId

- Every `_id` in a collection must have a unique value. The rest of the fields and values are not enforced in anyway.
 - So you can have a collection where all the documents have the same exact field-value pairs in each document where the only difference between them being the `_id` field.
 - Likewise, you can have a collection where each document is distinctly different from the other that they don't have the same shape or any field names in common.

3. Inserting New Documents - ObjectId

ObjectId()

Default value for the `_id` field unless otherwise specified.

Example:

```
"_id": ObjectId("5ec5f1b710ca9222e6a46cab")
```

The underscore ID doesn't have to have the type `ObjectId`. If you already know of unique values that you can use for each document, you can use those values in the underscore ID field instead.

Example:

```
"_id": "710ca922"
```

```
"_id": "101-EXG-27"
```

3. ObjectId (Quiz)

How does the value of `_id` get assigned to a document?

- It is automatically generated as an ObjectId type value.
- You can select a non ObjectId type value when inserting a new document, as long as that value is unique to this document.

3. Create - MQL: insert

Create -
MQL: insert

3. Inserting New Documents - insert() and errors

insert method inserts a new document into a collection

Syntax

```
db.<collection name>.insert(<document>)
```

Common errors

- duplicate key error collection
 - occurs if you try to insert two documents with the same _id

3. Insert Errors

- MongoDB can store duplicate documents in the same collection, as long as their `_id` values are different.
- If a document is inserted without a provided `_id` value, then the `_id` field and value will be automatically generated for the inserted document before insertion.

3. Create - Option: ordered

Create -
Option: ordered

3. Inserting New Documents - insert() order

We may insert multiple documents by using an array.

Syntax

```
db.<collection name>.insert([<document>,<document>,<document>])
```

Example:

```
db.inspections.insert([ {test:1}, {test:2}, {test:3} ])
```

3. Inserting New Documents - insert() order

By default, if you attempt to add two documents with same `_id` an error is thrown because of the duplication, and all the remaining inserts fails thereafter

Example:

```
db.inspections.insert([ {test:1, _id:1}, {test:2, _id:1}, {test:3, _id:3} ])
```

Use `{"ordered": false}` to disable the default ordered insert

To ensure all documents with unique `_ids` are added, then pass an options object to set ordered false.

Example:

```
db.inspections.insert([ {test:1, _id:1},{test:2, _id:1},{test:3, _id:3} ], {"ordered": false})
```

3. Inserting New Documents - insert() order

Important:

if a collection does not exist yet for the document to be inserted into, then mongo will make it. This can cause issues if the collection name was misspelled, as no error will be thrown

Collections and databases are created when they are being used.

3. Insert Order (Quiz)

Which of the following commands will successfully insert 3 new documents into an empty pets collection?

```
db.pets.insert([{"_id": 1, "pet": "cat"}, {"_id": 1, "pet": "dog"}, {"_id": 3, "pet": "fish"}, {"_id": 4, "pet": "snake"}], {"ordered": false })
```

```
db.pets.insert([{"_id": 1, "pet": "cat"}, {"_id": 1, "pet": "dog"}, {"_id": 3, "pet": "fish"}, {"_id": 4, "pet": "snake"}], {"ordered": true })
```

```
db.pets.insert([{"_id": 1, "pet": "cat"}, {"_id": 2, "pet": "dog"}, {"_id": 3, "pet": "fish"}, {"_id": 3, "pet": "snake"}])
```

```
db.pets.insert([{"pet": "cat"}, {"pet": "dog"}, {"pet": "fish"}])
```

3. Insert Order (Quiz)

Which of the following commands will successfully insert 3 new documents into an empty pets collection?

```
db.pets.insert([{\n  "_id": 1, "pet": "cat"\n},\n\n  {\n    "_id": 1, "pet": "dog"\n  },\n\n  {\n    "_id": 3, "pet": "fish"\n  },\n\n  {\n    "_id": 4, "pet":\n    "snake" }], {\n  "ordered": false })
```



```
db.pets.insert([{\n  "_id": 1, "pet": "cat"\n},\n\n  {\n    "_id": 1, "pet": "dog"\n  },\n\n  {\n    "_id": 3, "pet": "fish"\n  },\n\n  {\n    "_id": 4, "pet":\n    "snake" }], {\n  "ordered": true })
```



```
db.pets.insert([{\n  "_id": 1, "pet": "cat"\n},\n\n  {\n    "_id": 2, "pet": "dog"\n  },\n\n  {\n    "_id": 3, "pet": "fish"\n  },\n\n  {\n    "_id": 3, "pet":\n    "snake" }])
```



```
db.pets.insert([{\n  "pet": "cat" }, {\n  "pet": "dog" },\n\n  {\n    "pet": "fish" }])
```



3. Update

Update

3. Updating Documents (Quiz)

Problem:

MongoDB has a flexible data model, which means that you can have fields that contain documents, or arrays as their values. Select any **invalid** MongoDB documents from the given choices:

```
{ "_id": 1,
  "pet": "cat",
  "attributes": [ {
    "coat": "fur",

    "type": "soft" },
                  {
    "defense": "claws",

    "location": "paws",

    "nickname": "murder
mittens" } ],
  "name": "Furball" }
```

```
{ "_id": 1,
  "pet": "cat",
  "fur": "soft",
  "claws": "sharp",
  "name": "Furball" }
```

```
{ "_id": 1,
  "pet": "cat",
  "attributes": {
    "coat": "soft fur",

    "paws": "cute but
deadly" },
  "name": "Furball" }
```

None of the above.

3. Updating Documents (Quiz)

Problem:

MongoDB has a flexible data model, which means that you can have fields that contain documents, or arrays as their values. Select any **invalid** MongoDB documents from the given choices:

```
{ "_id": 1,
  "pet": "cat",
  "attributes": [ {
    "coat": "fur",

    "type": "soft" },
                  {
    "defense": "claws",

    "location": "paws",

    "nickname": "murder
mittens" } ],
  "name": "Furball" }
```

```
{ "_id": 1,
  "pet": "cat",
  "fur": "soft",
  "claws": "sharp",
  "name": "Furball" }
```

```
{ "_id": 1,
  "pet": "cat",
  "attributes": {
    "coat": "soft fur",

    "paws": "cute but
deadly" },
  "name": "Furball" }
```

None of the above.



3. Updating Documents - MQL

Update methods alter the value of a field in a document.

Syntax (updateMany):

```
db.<collection name>.updateMany(<query>, <update operator>)
```

Syntax (updateOne):

```
db.<collection name>.updateOne(<query>, <update operator>)
```

3. Updating Documents - MQL

One

`updateOne()`

`findOne()`



Many

`updateMany()`

`find()`



3. Updating Documents - MQL

Update operators

```
{"$inc": {"pop": 10, "<field2>": <increment value>, ... }}
```

increments field value by a specified amount.

```
{"$set": {"pop": 17630, "<field2>": <new value>, ... }}
```

sets field value to a new specified value.

```
{ $push: { <field1>: <value1>, ... } }
```

adds an element to an array field.

3. Updating Documents - MQL

Example 1:

```
db.zips.updateMany( { "city": "HUDSON" }, { "$inc": { "pop": 10 } } )
```

Example 2:

```
db.zips.updateOne({ "zip": "12534" }, { "$set": { "population": 17630 } })
```

Example 3:

[illegible]

3. Updating Documents - (Quiz)

Problem:

Given a **pets** collection where each document **has the following structure** and fields:

```
{
  "_id": ObjectId("5ec414e5e722bb1f65a25451"),
  "pet": "wolf",
  "domestic?": false,
  "diet": "carnivorous",
  "climate": ["polar", "equatorial", "continental", "mountain"]
}
```

Which of the following commands will add new fields to the updated documents?

```
db.pets.updateMany({ "pet": "cat" }, { "$set": { "climate": "continental" } })
```

```
db.pets.updateMany({ "pet": "cat" }, { "$set": { "type": "dangerous", "look": "adorable" } })
```

```
db.pets.updateMany({ "pet": "cat" }, { "$push": { "climate": "continental", "look": "adorable" } })
```

```
db.pets.updateMany({ "pet": "cat" }, { "$set": { "domestic?": true, "diet": "mice" } })
```

3. Updating Documents - (Quiz)

Problem:

Given a **pets** collection where each document **has the following structure** and fields:

```
{
  "_id": ObjectId("5ec414e5e722bb1f65a25451"),
  "pet": "wolf",
  "domestic?": false,
  "diet": "carnivorous",
  "climate": ["polar", "equatorial", "continental", "mountain"]
}
```

Which of the following commands will add new fields to the updated documents?

```
db.pets.updateMany({ "pet": "cat" }, { "$set": { "climate": "continental" } })
```

```
db.pets.updateMany({ "pet": "cat" }, { "$set": { "type": "dangerous", "look": "adorable" } })
```

```
db.pets.updateMany({ "pet": "cat" }, { "$push": { "climate": "continental", "look": "adorable" } })
```

```
db.pets.updateMany({ "pet": "cat" }, { "$set": { "domestic?": true, "diet": "mice" } })
```


3. Delete

Delete

3. Deleting Documents & Collections - MQL

delete/drop methods removes documents from a collection or removes collection.

Syntax (updateMany): removes multiple documents from a collection based on query

```
db.<collection name>.deleteMany(<query>)
```

Syntax (updateOne): removes single document from a collection based on query

```
db.<collection name>.deleteOne(<query>)
```

Syntax (drop): removes a collection from database

```
db.<collection name>.drop()
```

3. Deleting Documents & Collections - MQL

One

```
deleteOne("_id":11)
```

```
updateOne("_id":11)
```

```
findOne("_id":11)
```



Many

```
deleteMany()
```

```
updateMany()
```

```
find()
```



3. Deleting Documents & Collections - MQL

Example 1:

```
db.inspections.deleteMany({ "test": 1 })
```

Delete all the documents that have test field equal to 1.

Example 2:

```
db.zips.updateOne({ "zip": "12534" }, { "$set": { "population": 17630 } })
```

Delete one document that has test field equal to 3.

Example 3:

```
db.inspection.drop()
```

Drop the inspection collection

3. Deleting Documents & Collections - MQL (Quiz 1)

Problem:

Suppose you get rid of all the collections in a database. Does removing all collections in a database also remove the database?

YES	NO
-----	----

3. Deleting Documents & Collections - MQL (Quiz 1)

Problem:

Suppose you get rid of all the collections in a database. Does removing all collections in a database also remove the database?

YES

NO

3. Deleting Documents & Collections - MQL (Quiz 2)

Problem:

Which of the following commands will delete a collection named villains?

A	<code>db.villains.delete()</code>
B	<code>db.villains.drop()</code>
C	<code>db.villains.dropAll()</code>

3. Deleting Documents & Collections - MQL (Quiz 2)

Problem:

Which of the following commands will delete a collection named villains?

A	<code>db.villains.delete()</code>
B	<code>db.villains.drop()</code>
C	<code>db.villains.dropAll()</code>

4. Advanced CRUD Operations

Part 4

Advanced CRUD Operations

4. Advanced CRUD Operations

- **Query Operators - Comparisons**
- **Query Operators - Logic**
- **Query Operators - Expressions**
- **Query Operators - Arrays**
- **Query Operators - Arrays & Projection**
- **Query Operators - Arrays & Sub-Documents**

4. Review of CRUD Operators

MQL operators

Update Operators

Example: `$inc`, `$set`, `$unset`

Enable us to modify data in the database

Query Operators

Provide additional ways to locate data within the database

\$ has multiple uses

Precedes MQL operators

Precedes Aggregation pipeline stages

Allows Access to Field Values

We can use the simple query/update operators along with more powerful operators common to the JS programming language like: comparisons, logicals, and array-based operations.

4. Query Operators - Comparison

Comparison Operators

4. Query Operators - Comparison

Comparison queries utilize the comparator operators common in most programming languages.

JS Comparisons:

- greater than or equal to ($\geq$)
- greater than ($>$)
- less than ($<$)
- less than or equal to ($\leq$)

SQL Syntax:

```
{ <field name>: { <operator>: <value> } }
```

4. Query Operators - Comparison

Comparison operators

`$eq` `=` Equal to

`$ne` `=` Not Equal to

`$gt` `>` Greater Than

`$lt` `<` Less Than

`$gte` `≥` Greater Than or Equal to

`$lte` `≤` Less Than or Equal to

```
{ <field>: { <operator>: <value> } }
```

4. Query Operators - Comparison

Comparison operators

Query operators provide additional ways to locate data within the database.

Comparison operators specifically allow us to find data within a certain range.

```
{ <field>: { <operator>: <value> } }
```

`$eq` is used as the default operator when an operator is not specified.

4. Query Operators - Comparison - Examples

Example 1:

```
db.trips.find({ "tripduration": { "$lte" : 70 }, "usertype": { "$ne": "Subscriber" } })
```

Find all documents where the tripduration was less than or equal to 70 seconds and the usertype was not Subscriber:

Example 2:

```
db.trips.find({ "tripduration": { "$lte" : 70 }, "usertype": { "$eq": "Customer" } })
```

Find all documents where the tripduration was less than or equal to 70 seconds and the usertype was Customer using a redundant equality operator

Example 3:

```
db.trips.find({ "tripduration": { "$lte" : 70 }, "usertype": "Customer" })
```

Find all documents where the tripduration was less than or equal to 70 seconds and the usertype was Customer using the implicit equality operator

4. Query Operators - Comparison - (Quiz 1)

Problem:

How would you define a comparison query to find all documents that have fewer than 1000 people listed in the pop field for the zips collection?

4. Query Operators - Comparison - (Quiz 1)

Problem:

How would you define a comparison query to find all documents that have fewer than 1000 people listed in the pop field for the zips collection?

```
db.zips.find({ "pop": { "$lt": 1000 } }).count()
```

4. Query Operators - Comparison - (Quiz 2)

Problem:

How would you define a comparison query to find the difference between the number of people born in 1998 and the number of people born after 1998 from a collection named 'trips'?

4. Query Operators - Comparison - (Quiz 2)

Problem:

How would you define a comparison query to find the difference between the number of people born in 1998 and the number of people born after 1998 from a collection named 'trips'?

```
db.trips.find({ "birth year": { "$gt": 1998 } }).count()  
db.trips.find({ "birth year": 1998 }).count()
```

Use the \$gt instead of \$gte operator to exclude all 1998 births, and then see how many people were born in 1998 by using implicit equality, then subtract the two values to get the difference

4. Query Operators - Comparison - (Quiz 3)

Problem:

From the routes collection find out which of the following statements will return all routes that have at least one stop in them? Select all answers that are correct.

<i>A</i>	<code>db.routes.find({ "stops": { "\$ne": 0 } }).pretty()</code>
<i>B</i>	<code>db.routes.find({ "stops": { "\$gte": 0 } }).pretty()</code>
<i>C</i>	<code>db.routes.find({ "stops": { "\$lt": 0 } }).pretty()</code>
<i>D</i>	<code>db.routes.find({ "stops": { "\$gt": 0 } }).pretty()</code>

4. Query Operators - Comparison - (Quiz 3)

Problem:

From the routes collection find out which of the following statements will return all routes that have at least one stop in them? Select all answers that are correct.

A	<code>db.routes.find({ "stops": { "\$ne": 0 } }).pretty()</code>
B	<code>db.routes.find({ "stops": { "\$gte": 0 } }).pretty()</code>
C	<code>db.routes.find({ "stops": { "\$lt": 0 } }).pretty()</code>
D	<code>db.routes.find({ "stops": { "\$gt": 0 } }).pretty()</code>

4. Query Operators - Logic

Logical Operators

4. Query Operators - Logic

Logical queries utilize the logical operators common in most programming languages.

JS Logical operations:

- and (&&)
- or (||)
- not (!)

SQL Syntax:

```
{ <operator>: [<query>,<query>,<query>] }
```


4. Query Operators - Logic - MQL

Logic operators

- `$and` Match **all** of the specified query clauses
- `$or` **At least one** of the query clauses is matched
- `$nor` **Fail to match** both given clauses
- `$not` **Negates** the query requirement

4. Query Operators - Logic - MQL

Logic operators

`$and`

`$or` `{<operator> : [{statement1},{statement2},...]}`

`$nor`

`$not` `{$not: {statement}}`

4. Query Operators - Logic - \$and

Implicit \$and

\$and is used as the default operator when an operator is not specified.

```
{sector : "Mobile Food Vendor - 881",  result: "Warning"}
```

Is the same as:

```
{"$and": [{sector : "Mobile Food Vendor - 881"}, {result:"Warning"}]}
```

4. Query Operators - Logic - \$and

Implicit \$and

Find which student ids are > 25 and < 100 in the `sample_training.grades` collection.

```
{"$and": [{"student_id": {"$gt": 25}}, {"student_id": {"$lt": 100}}]}
```

Is the same as

```
{"student_id": {"$gt": 25}, "student_id": {"$lt": 100}}
```

Better

```
{"student_id": {"$gt": 25, "$lt": 100}}
```

4. Query Operators - Logic - \$and

Explicit \$and

When you need to include the same operator more than once in a query

Using the **routes** collection find out how many CR2 and A81 airplanes come through the KZN airport?

```
{"$or" : [{dst_airport : "KZN"}, {src_airport : "KZN"}]}
```

and

```
{"$or" : [{airplane : "CR2"}, {airplane : "A81"}]}
```

4. Query Operators - Logic - Summary

Logic operators

Logic operators allow us to be more granular in our search for data.

Syntax

```
{ "$<operator>": [{ <clause1> }, {<clause2>}, ... ] }
```

Syntax for **\$not**:

```
{ $not: {<clause>} }
```

\$and is used as the default operator when an operator is not specified.

Explicitly use **\$and** when you need to include the same operator more than once in a query.

4. Query Operators - Logic - Examples

Example 1:

```
db.routes.find({ "$and": [ { "$or" :[ { "dst_airport": "KZN" },
                                     { "src_airport": "KZN" }
                                   ] },
                        { "$or" :[ { "airplane": "CR2" },
                                   { "airplane": "A81" } ] }
                      ]})
```

Find all documents where airplanes CR2 or A81 left or landed in the KZN airport from the routes collection:

4. Query Operators - Logic - (Quiz 1)

Problem:

Define a logical query that finds how many businesses in the 'inspections' dataset have the inspection result "Out of Business" and belong to the "Home Improvement Contractor - 100" sector?

4. Query Operators - Logic - (Quiz 1)

Problem:

Define a logical query that finds how many businesses in the 'inspections' dataset have the inspection result "Out of Business" and belong to the "Home Improvement Contractor - 100" sector?

```
db.inspections.find({ "result": "Out of Business",  
                      "sector": "Home Improvement Contractor - 100" }).count()
```

This is a trick question because there is no need to explicitly use a logic operator, you can get the result by using an implicit \$and and specifying the query conditions separated by a comma. However, you will still get the right answer if you used an explicit \$and operator to reach this answer.

4. Query Operators - Logic - (Quiz 2)

Problem:

Which is **the most succinct** query to return all documents from the `inspections` collection where the inspection date is either "Feb 20 2015", or "Feb 21 2015" and the company is **not** part of the "Cigarette Retail Dealer - 127" sector?

```
db.inspections.find(
  { "$or": [ { "date": "Feb 20 2015" },
             { "date": "Feb 21 2015" } ],
    "sector": { "$ne": "Cigarette Retail Dealer - 127" } })
```

```
db.inspections.find(
  { "$and": [
    { "$or": [
      { "date": "Feb 20 2015" },
      { "date": "Feb 21 2015" } ] },
    { "sector": { "$ne": "Cigarette Retail Dealer - 127" } } ]
  })
```

```
db.inspections.find(
  { "$or": [ { "date": "Feb 20 2015" },
             { "date": "Feb 21 2015" } ] },
  { "$not": { "sector": "Cigarette Retail Dealer - 127" } })
```

4. Query Operators - Logic - (Quiz 2)

Problem:

Which is **the most succinct** query to return all documents from the `inspections` collection where the inspection date is either "Feb 20 2015", or "Feb 21 2015" and the company is **not** part of the "Cigarette Retail Dealer - 127" sector?

```
db.inspections.find(
  { "$or": [ { "date": "Feb 20 2015" },
             { "date": "Feb 21 2015" } ],
    "sector": { "$ne": "Cigarette Retail Dealer - 127" } })
```

```
db.inspections.find(
  { "$and": [
    { "$or": [
      { "date": "Feb 20 2015" },
      { "date": "Feb 21 2015" } ] },
    { "sector": { "$ne": "Cigarette Retail Dealer - 127" } } ]
  })
```

```
db.inspections.find(
  { "$or": [ { "date": "Feb 20 2015" },
             { "date": "Feb 21 2015" } ],
    "$not": { "sector": "Cigarette Retail Dealer - 127" } })
```

This query accounts for both dates that we are looking for and excludes all documents where the sector is "Cigarette Retail Dealer - 127" by using the `$ne` not equal comparison operator.

This statement will return the correct documents, but it is unnecessarily using the `$and` operator and thus can not be the best way to get the required documents.

While this query looks for the right dates and correctly uses the `$or` operator, it is incorrectly using the `$not` logic operator. `$not` negates a logical expression, and in this case, will result in an error.

4. Query Operators - Logic - (Quiz 3)

Problem:

Define a logical query that finds how many zips in the zips dataset are neither overpopulated or underpopulated? In this case, we consider population of more than 1,000,000 to be overpopulated and less than 5,000 to be under-populated.

4. Query Operators - Logic - (Quiz 3)

Problem:

Define a logical query that finds how many zips in the zips dataset are neither overpopulated or underpopulated? In this case, we consider population of more than 1,000,000 to be overpopulated and less than 5,000 to be under-populated.

```
db.zips.find({ "$nor": [ { "pop": { "$lt": 5000 } },  
                          { "pop": { "$gt": 1000000 } } ] } ).count()
```

The best operator for this case is \$nor, it excludes both cases from consideration and we are left with the remaining set.

4. Query Operators - Logic - (Quiz 4)

Problem:

Define a logical query that finds how many companies in the `companies` dataset were either founded in **2004** and either have the *social* `category_code` or *web* `category_code`, or were founded in the month of **October** and also either have the *social* `category_code` or *web* `category_code`?

4. Query Operators - Logic - (Quiz 4)

Problem:

Define a logical query that finds how many companies in the companies dataset were either founded in **2004** and either have the *social* **category_code** or *web* **category_code**, or were founded in the month of **October** and also either have the *social* **category_code** or *web* **category_code**?

```
db.companies.find({ "$and": [
    { "$or": [ { "founded_year": 2004 }, { "founded_month": 10 } ] },
    { "$or": [ { "category_code": "web" }, { "category_code": "social" } ] }
]
}).count()
```

In this case, you have to use an explicit \$and. There are two or statements, and to get all the companies that match the required specification both or statements have to be true at the same time.

4. Query Operators - Expression

Expressive
Operators

4. Query Operators - Expression

The expressive query operator allows for more than one simple operation with aggregation syntax. Expressive query operators also allow for us to access field values from documents directly for comparison to each other.

4. Query Operators - Expression - operator

Expressive \$expr

\$expr allows the use of aggregation expressions within the query language

```
{ $expr: { <expression> } }
```

\$expr allows us to use variables and conditional statements

Woo-hoo!

Can we compare fields **within the same document** to each other?

4. Query Operators - Expression - \$ symbol

\$



\$ denotes the use of an operator

\$ addresses the field value

4. Query Operators - Expression - accessing fields

\$ addressing the field value

```
{"$expr" : {"$eq": ["$start station id", "$end station id"]}}
```

```
{
  "_id": "572bb8222b288919b68abf70",
  "tripduration": 110,
  "start station id": 439,
  "start station name": "E 4 St & 2 Ave",
  "end station id": 439,
  "end station name": "E 4 St & 2 Ave",
  ...
  "start station location":
    { "type": "Point",
      "coordinates": [-73.98978041,
                     40.7262807]
    },
  ...
}
```

4. Query Operators - Expression - accessing fields

\$ addressing the field value

```
{"$expr" : {"$eq": [{"$start station name", "$end station id"]}}
```

```
{
  "_id": "572bb8222b288919b68abf70",
  "tripduration": 110,
  "start station id": 439,
  "start station name": "E 4 St & 2 Ave",
  "end station id": 439,
  "end station name": "E 4 St & 2 Ave",
  ...
  "start station location":
    { "type": "Point",
      "coordinates": [-73.98978041,
                      40.7262807]
    },
  ...
}
```

4. Query Operators - Expression - aggregation syntax

A closer look

```
{ "$expr": {  
  "$and": [  
    { "$gt": [ "$tripduration", 1200 ] },  
    { "$eq": [ "$end station id", "$start station id" ] }  
  ]  
}
```

MQL syntax: { <field>: { <operator>: <value> } }

Aggregation syntax: { <operator>: { <field>, <value> } }

4. Query Operators - Expression - summary

\$expr: Expressive Query Operator

Allows for more complex queries and for comparing fields within a document.

The `$` can be used to access the field value

Syntax for comparison operators using aggregation:

```
{ <operator>: { <field>, <value> } }
```

4. Query Operators - Expression - Examples

Example 1:

```
db.trips.find({ "$expr": { "$eq": [ "$end station id", "$start station id" ] }  
}).count()
```

Find all documents where the trip started and ended at the same station

Example 2:

```
db.trips.find({ "$expr": { "$and": [ { "$gt": [ "$tripduration", 1200 ] },  
    { "$eq": [ "$end station id", "$start station id" ] }  
    ] } }).count()
```

Find all documents where the trip lasted longer than 1200 seconds, and started and ended at the same station:

4. Query Operators - Expression - (Quiz 1)

Problem:

What are some of the uses for the \$ sign in MQL? Select all that apply.

☐ \$ changes the data type of the given value to a monetary denomination.

☐ \$ denotes an operator.

☐ \$ signifies that you are looking at the value of that field rather than the field name.

4. Query Operators - Expression - (Quiz 1)

Problem:

What are some of the uses for the \$ sign in MQL? Select all that apply.

☐ \$ changes the data type of the given value to a monetary denomination.

☐ \$ denotes an operator.

☐ \$ signifies that you are looking at the value of that field rather than the field name.

4. Query Operators - Expression - (Quiz 2)

Problem:

Which of the following statements will find all the companies that have more employees than the year in which they were founded? Select ALL that apply.

A	<pre>db.companies.find({ "\$expr": { "\$gt": ["\$number_of_employees", "\$founded_year"] } })</pre>
B	<pre>db.companies.find({ "number_of_employees": { "\$gt": "\$founded_year" } }).count()</pre>
C	<pre>db.companies.find({ "\$expr": { "\$gt": ["\$founded_year", "\$number_of_employees"] } })</pre>
D	<pre>db.companies.find({ "\$expr": { "\$lt": ["\$founded_year", "\$number_of_employees"] } })</pre>

4. Query Operators - Expression - (Quiz 2)

Problem:

Which of the following statements will find all the companies that have more employees than the year in which they were founded? Select ALL that apply.

A	<pre>db.companies.find({ "\$expr": { "\$gt": ["\$number_of_employees", "\$founded_year"] } })</pre>
B	<pre>db.companies.find({ "number_of_employees": { "\$gt": "\$founded_year" } }).count()</pre>
C	<pre>db.companies.find({ "\$expr": { "\$gt": ["\$founded_year", "\$number_of_employees"] } })</pre>
D	<pre>db.companies.find({ "\$expr": { "\$lt": ["\$founded_year", "\$number_of_employees"] } })</pre>

Use \$expr operator to compare the 2 field values in each document. The \$expr operator requires aggregation syntax for the comparison operator.

Wrong syntax, it looks at every number of employees field and compares it to some specific numeric value that is not specified.

This query is looking for when the founded year is greater than the number of employees which is the opposite of what we are looking for.

Use \$expr operator to compare the 2 field values in each document. The \$expr operator requires aggregation syntax for the comparison operator.

4. Query Operators - Expression - (Quiz 3)

Problem:

Write a Expressive Query that finds how many companies in the `companies` collection have the same `permalink` as their `twitter_username`?

4. Query Operators - Expression - (Quiz 3)

Problem:

Write a Expressive Query that finds how many companies in the `companies` collection have the same `permalink` as their `twitter_username`?

```
db.companies.find({ "$expr": { "$eq": [ "$permalink", "$twitter_username" ] }  
}).count()
```

We are comparing the values of the two fields within the same document to see if they are equal using the \$ to access field values, and the aggregation syntax to use the \$eq operator as we saw in the lesson.

4. Query Operators - Array

Array Operators

4. Query Operators - Array

We've queried documents by fields that have string, numeric, & boolean values. Let's look at more complex elements such as arrays.

We've seen how update has a \$push operator which adds a value into an array.

4. Query Operators - Array

Array operators

`$push`

Allows us to add an element to an array.

`$push`

Turns a field into an array field if it was previously a different type.

4. Query Operators - Array - \$all, \$size

Array operators

```
{<array field> : { "$size": <number>}}
```

Returns a cursor with all documents where the specified array field is exactly the given length.

```
{<array field> : { "$all": <array>}}
```

Returns a cursor with all documents in which the specified array field contains all the given elements regardless of their order in the array.

Querying an array field using

An array returns only exact array matches

A single element will return all documents where the specified array field contains this given element.

4. Query Operators - Array - Examples

Example 1:

```
db.listingsAndReviews.find({ "amenities": {
    "$size": 20,
    "$all": [ "Internet", "Wifi", "Kitchen",
              "Heating", "Family/kid friendly",
              "Washer", "Dryer", "Essentials",
              "Shampoo", "Hangers",
              "Hair dryer", "Iron",
              "Laptop friendly workspace" ]
    }
})
```

Find all documents with exactly 20 amenities which include all the amenities listed in the query array

4. Query Operators - Array - (Quiz 1)

Problem:

Write an Array query that finds the the listings in the `listingsAndReviews` dataset that accommodates more than 6 people and has exactly 50 reviews?

```
db.listingsAndReviews.find({ "reviews": { "$size":50 },  
                           "accommodates": { "$gt":6 }})
```

We can use the \$size operator to select only the documents that have exactly 50 elements in the reviews field. This query can also run in Atlas so that it is easier to see the value of the name field right away.

4. Query Operators - Array - (Quiz 2)

Problem:

Using the `listingsAndReviews` collection find out how many documents have the `"property_type"` of `"House"`, and include `"Changing table"` as one of the `"amenities"`?

```
db.listingsAndReviews.find({ "property_type": "House",  
                             "amenities": "Changing table" }).count()
```

There is no need to use any array operators in this scenario. The default MQL behavior is to return documents in which the `"amenities"` field contains the element `"Changing table"`.

4. Query Operators - Array - (Quiz 3)

Problem:

Which of the following queries will return all listings that have "Free parking on premises", "Air conditioning", and "Wifi" as part of their **amenities**, and have at least 2 bedrooms in the **listingsAndReviews** collection?

```
db.listingsAndReviews.find(
  { "amenities": [ "Free parking on premises", "Wifi", "Air conditioning" ] },
  "bedrooms": { "$gte": 2 } } ).pretty()
```

```
db.listingsAndReviews.find(
  { "amenities": "Free parking on premises",
    "amenities": "Wifi",
    "amenities": "Air conditioning",
    "bedrooms": { "$gte": 2 } } ).pretty()
```

```
db.listingsAndReviews.find(
  { "amenities":
    { "$all": [ "Free parking on premises", "Wifi", "Air conditioning" ] } },
  "bedrooms": { "$lte": 2 } } ).pretty()
```

```
db.listingsAndReviews.find(
  { "amenities":
    { "$all": [ "Free parking on premises", "Wifi", "Air conditioning" ] } },
  "bedrooms": { "$gte": 2 } } ).pretty()
```

4. Query Operators - Array - (Quiz 3)

Problem:

Which of the following queries will return all listings that have "Free parking on premises", "Air conditioning", and "Wifi" as part of their **amenities**, and have at least 2 bedrooms in the **listingsAndReviews** collection?

<pre>db.listingsAndReviews.find({ "amenities": ["Free parking on premises", "Wifi", "Air conditioning"] }, "bedrooms": { "\$gte": 2 } }).pretty()</pre>	Because the amenities are listed in an array without any additional operators, MongoDB will look for documents that are an exact match. Only documents in which the amenities field looks exactly like the one in the query will be returned.
<pre>db.listingsAndReviews.find({ "amenities": "Free parking on premises", "amenities": "Wifi", "amenities": "Air conditioning", "bedrooms": { "\$gte": 2 } }).pretty()</pre>	A query should be a valid JSON document. A valid JSON document cannot have multiple identical keys so MongoDB will go through each query requirement overwriting requirements with identical field names until it reaches the last mention of a field in this query.
<pre>db.listingsAndReviews.find({ "amenities": { "\$all": ["Free parking on premises", "Wifi", "Air conditioning"] }, "bedrooms": { "\$lte": 2 } }).pretty()</pre>	The array field query is composed correctly, but the bedrooms requirement isn't met because \$lte will return all listings with at most two bedrooms instead of at least two.
<pre>db.listingsAndReviews.find({ "amenities": { "\$all": ["Free parking on premises", "Wifi", "Air conditioning"] }, "bedrooms": { "\$gte": 2 } }).pretty()</pre>	Use the \$all array operator in order to get all documents where the amenities array contains these three elements regardless of their order. To get the desired number of bedrooms use \$gte, since it includes 2, but also shows other listings with more bedrooms.

4. Query Operators - Array & Projection

Array Operators & Projection

4. Query Operators - Array & Projection

The MQL `find()` method may receive a second parameter, a **projection** object.

A **projection** object defines which document fields will either be or not be part of the resulting cursor.

Syntax:

```
db.<collection>.find({<query>},{<project>})
```

4. Query Operators - Array & Projection

Projection Syntax

```
db.<collection>.find({ <query> }, { <projection> })
```

1 - include the field

0 - exclude the field

Use only 1s or only 0s

```
db.<collection>.find({ <query> }, { <field1>: 1, <field2>: 1 })
```

or

```
db.<collection>.find({ <query> }, { <field1>: 0, <field2>: 0 })
```

exception:

```
db.<collection>.find({ <query> }, { <field1>: 1, "_id": 0 })
```

4. Query Operators - Array & Projection

Projection

```
db.listingsAndReviews.find(  
  {"amenities": {"$size": 20,  
                 "$all": ["Internet", "Wifi", "Kitchen",  
                           "Heating", ...]}},  
  {"price": 1, "address": 1}).pretty()
```

Only include the **price** and **address** fields in the cursor result.

4. Query Operators - Array & Projection

`$elemMatch` matches documents that contain an array field with at least one element that matches the specified query criteria

Syntax:

```
{<field>: { "$elemmatch": {<field>:<value>} } }
```

4. Query Operators - Array & Projection

Projection and \$elemMatch

```
db.<collection>.find({ <query> }, { <projection> })
```

Specifies which fields should or should not be included in the result cursor.

Do **not** combine 1s and 0s in a projection

- Except for { "_id: 0", <field>: 1 }

```
{<field> : { "$elemMatch": { <field>: <value> } }}
```

Matches documents that contain an array field with at least one element that matches the specified query criteria.

or

Projects only the array elements with at least one element that matches the specified criteria.

4. Query Operators - Array & Projection - Examples

Example 1:

```
db.listingsAndReviews.find({ "amenities":  
  { "$size": 20, "$all": [ "Internet", "Wifi", "Kitchen", "Heating",  
    "Family/kid friendly", "Washer", "Dryer",  
    "Essentials", "Shampoo", "Hangers",  
    "Hair dryer", "Iron",  
    "Laptop friendly workspace" ] } },  
  {"price": 1, "address": 1}).pretty()
```

Find all documents with exactly 20 amenities which include all the amenities listed in the query array, and display their price and address:

4. Query Operators - Array & Projection - Examples

Example 2:

```
db.listingsAndReviews.find({ "amenities": "Wifi" },  
                           { "price": 1, "address": 1, "_id": 0 }).pretty()
```

Find all documents that have Wifi as one of the amenities only include price and address in the resulting cursor:

4. Query Operators - Array & Projection - Examples

Example 3:

```
db.grades.find({ "class_id": 431 },  
               { "scores": { "$elemMatch": { "score": { "$gt": 85 } } }  
               }).pretty()
```

Find all documents where the student in class 431 received a grade higher than 85 for any type of assignment:

4. Query Operators - Array & Projection - Examples

Example 4:

```
db.grades.find({ "scores": { "$elemMatch": { "type": "extra credit" } }  
}).pretty()
```

Find all documents where the student had an extra credit score:

4. Query Operators - Array & Projection - (Quiz 1)

Problem:

Define a Array query that finds how many companies in the `companies` collection have offices in the city of Seattle?

4. Query Operators - Array & Projection - (Quiz 1)

Problem:

Define a Array query that finds how many companies in the `companies` collection have offices in the city of Seattle?

```
db.companies.find({ "offices": { "$elemMatch": { "city": "Seattle" } }  
}).count()
```

"offices" is an array that contains documents with the address information from each office. We use `$elemMatch` to return all documents in which an office array element contains the field `city` with the value `Seattle`.

4. Query Operators - Array & Projection - (Quiz 2)

Problem:

Which of the following queries will return only the names of companies from the **companies** collection that had exactly 8 funding rounds? (Choose the Best)

```
db.companies.find({ "funding_rounds": { "$size": 8 } },  
                  { "name": 1, "_id": 0 })
```

```
db.companies.find({ "funding_rounds": { "$size": 8 } },  
                  { "name": 0, "_id": 1 })
```

```
db.companies.find({ "funding_rounds": { "$size": 8 } }, { "name": 1 })
```

4. Query Operators - Array & Projection - (Quiz 2)

Problem:

Which of the following queries will return only the names of companies from the **companies** collection that had exactly 8 funding rounds? (Choose the Best)

<pre>db.companies.find({ "funding_rounds": { "\$size": 8 } }, { "name": 1, "_id": 0 })</pre>	Since we are looking to only view the names of companies then we have to explicitly exclude the <code>_id</code> in the projection.
<pre>db.companies.find({ "funding_rounds": { "\$size": 8 } }, { "name": 0, "_id": 1 })</pre>	When using a zero in projection we exclude the specified field, so if this query worked it would return all fields except for the name field. This query, however, will return an error because it has a mix of inclusion and exclusion that is not allowed in projection.
<pre>db.companies.find({ "funding_rounds": { "\$size": 8 } }, { "name": 1 })</pre>	By default, the <code>_id</code> values will be included in the resulting cursor, but the question prompt asks to only view the company names. This query will return both <code>_id</code> and name values for matching documents, thus not satisfying the question prompt.

4. Query Operators - Array & Projection

Array Operators & Sub-Documents

4. Query Operators - Array & Sub-Documents

Querying arrays and sub-documents

MQL uses dot-notation to specify the address of nested elements in a document

To use dot-notation in arrays specify the position of the element in the array.

```
db.collection.find({"field 1.other field.also a field": "value"})
```

4. Query Operators - Array & Sub-Documents

Dot notation may be used in a query to access fields in a sub-document.

The diagram illustrates the use of dot notation in a MongoDB query and document structure. A green curved arrow points from the `location.type` field in the query to the `type` field in the document. Another green curved arrow points from the `location` field in the document to the `db.collection` label. A third green curved arrow points from the `location` field in the document to the `location.type` field in the query.

```
db.trips.findOne({"start station location.type": "Point"})
```

```
{  "_id": "572bb8222b288919b68abf70",
  ...
  "start station location" : {
    "type" : "Point",
    "coordinates" : [
      -73.97966069,
      40.74394314
    ]
  },
  ... }
```

`db.collection`

4. Query Operators - Array & Sub-Documents

Syntax for using dot notation in MQL queries.

```
db.collection.find({"field 1.other field.also a field": "value"})

{
  "_id": "572bb822abf70",
  "field 1" : {
    "some field" : "some number",
    "other field" : {
      "also a field" : "value",
      "field here" : "val too"
    }
  },
  "field 2" : "value 2",
  "field 3" : "value 3"
}
```

4. Query Operators - Array & Sub-Documents

Index numbers may be used in dot notation to access a specific position in an array from a MQL query

```
db.companies.find({ "relationships.0.person.last_name": "Zuckerberg"},  
                  { "name": 1 }).pretty()
```

0: position of the first array element

person: field name with a nested object as a value

last_name: field name within the "person" sub-document

"Zuckerberg": value that we are looking for

{ "name": 1 }: projection to only include the company name in the resulting cursor

4. Query Operators - Array & Sub-Documents

MQL Queries may use the \$regex operator for matching

CEOs named Mark listed first in the `relationships` array.

```
db.companies.find({ "relationships.0.person.first_name": "Mark",  
                    "relationships.0.title": { "$regex": "CEO" } },  
                  { "name": 1 }).pretty()
```

4. Query Operators - Array & Sub-Documents - (Quiz 1)

Problem:

Define a MQL Query that finds how many trips in the sample_training.trips collection started at stations that are to the west of the -74 longitude coordinate? *Note: Longitude decreases in value as you move west.*

4. Query Operators - Array & Sub-Documents - (Quiz 1)

Problem:

Write a MQL Query that finds how many trips in the sample_training.trips collection started at stations that are to the west of the -74 longitude coordinate? *Note: Longitude decreases in value as you move west.*

```
db.trips.find({ "start station location.coordinates": { "$lt": -74 } }).count()
```

The "start station location" has a sub-document that contains the coordinates array. To get to this coordinates array we must use dot-notation. We can issue a range query to find all documents in this longitude. The caveat is to remember that all trips take place in NYC so the latitude value in the coordinates array will always be positive, and we don't have to worry about it when issuing a range query like this.

4. Query Operators - Array & Sub-Documents - (Quiz 2)

Problem:

Write a MQL Query that finds how many inspections from the `inspections` collection were conducted in the city of NEW YORK?

4. Query Operators - Array & Sub-Documents - (Quiz 2)

Problem:

Write a MQL Query that finds how many inspections from the `inspections` collection were conducted in the city of NEW YORK?

```
db.inspections.find({ "address.city": "NEW YORK" }).count()
```

Here we need to use dot-notation to get to the "city" field and search for all documents where this field is equal to "NEW YORK".

4. Query Operators - Array & Sub-Documents - (Quiz 3)

Problem:

Which of the following queries will return the names and addresses of all listings from the **listingsAndReviews** collection where the first **amenity** in the list is "Internet"?

<pre>db.listingsAndReviews.find({ "amenities.\$0": "Internet" }, { "name": 1, "address": 1 }).pretty()</pre>	There is no need to add a \$ before stating the element position in this case. This query will not return any documents.
<pre>db.listingsAndReviews.find({ "amenities.0": "Internet" }, { "name": 1, "address": 1 }).pretty()</pre>	To reach the first element in an array field we need to use dot notation and the element's position which is zero for the first element in most arrays in most languages. For the projection part, we don't need to explicitly exclude the <code>_id</code> field because the question prompt isn't terribly strict about that.
<pre>db.listingsAndReviews.find({ "amenities.[0]": "Internet" }, { "name": 1, "address": 1 }).pretty()</pre>	Similarly, there is no need for square brackets around the position of the array element, and this query will also return zero documents.

5. Indexing & Aggregation Pipeline

Part 5

Indexing & Aggregation Pipeline

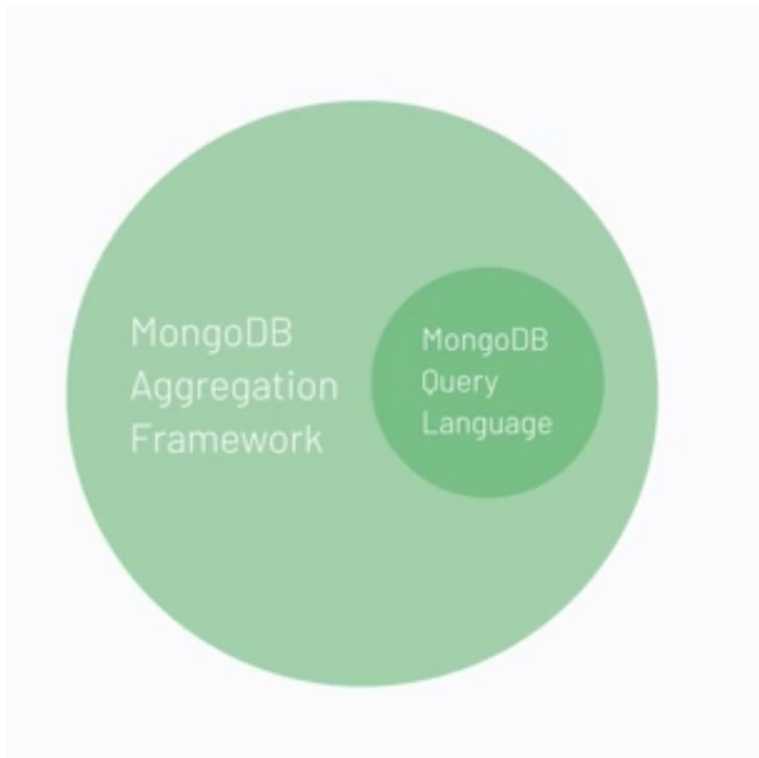
5. Indexing & Aggregation Pipeline

- **Aggregation Framework**
- **sort() & limit()**
- **Introduction to Indexes**
- **Introduction to Data Modeling**
- **Upsert - Update or Insert?**

5. Aggregation Framework

Aggregation Framework

5. Aggregation Framework



Aggregation Framework

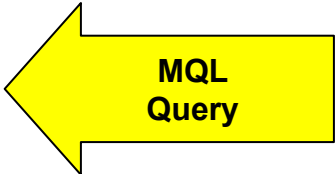
In its simplest form, another way to query data in MongoDB

5. Aggregation Framework - MQL vs. Aggregation

Syntax

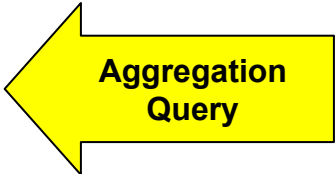
Find all documents that have Wifi as one of the amenities only includes price and address in the resulting cursor.

```
db.listingsAndReviews.find(  
  {"amenities": "Wifi"},  
  {"price": 1, "address": 1, "_id": 0}).pretty()
```



**MQL
Query**

```
db.listingsAndReviews.aggregate([  
  { $match: { "amenities": "Wifi" } },  
  { $project: { "price": 1, "address": 1, "_id": 0 } }  
])
```



**Aggregation
Query**

5. Aggregation Framework - Aggregation: Array of Queries

Aggregation framework defines an array of query actions.

Syntax

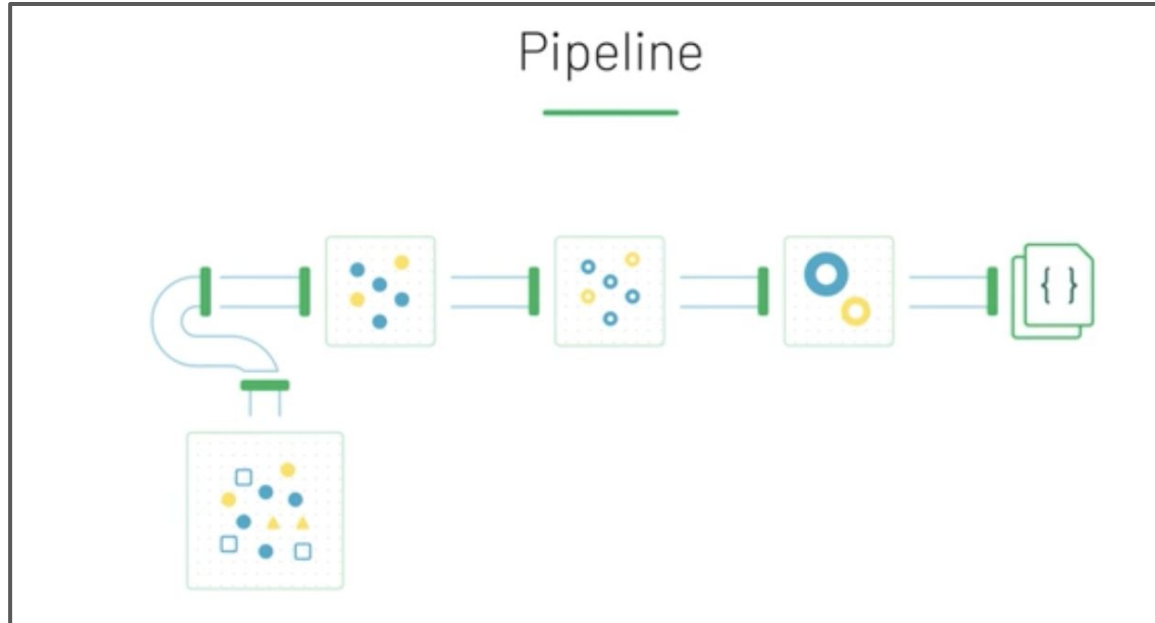
Find all documents that have Wifi as one of the amenities only includes price and address in the resulting cursor.

```
db.listingsAndReviews.aggregate(  
  { $match: { "amenities": "Wifi" } },  
  { $project: { "price": 1, "address": 1, "_id": 0 } }  
)
```

ARRAYS

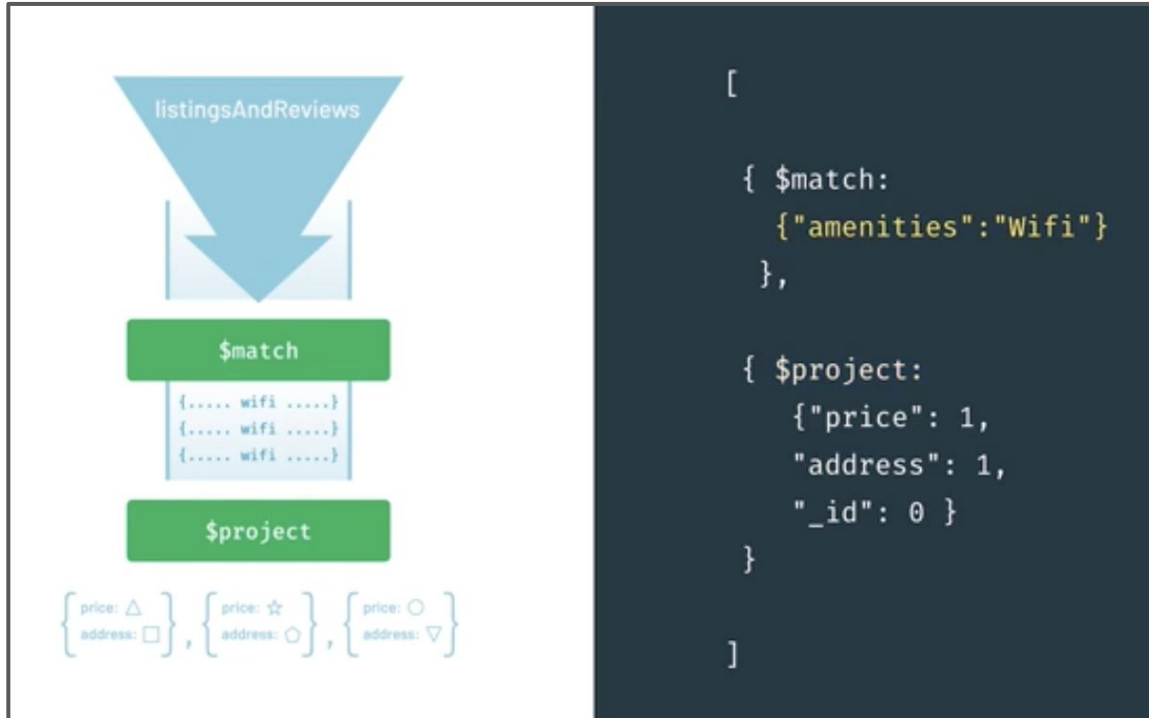
5. Aggregation Framework - Aggregation Pipeline

Aggregation framework works as a pipeline, passing the results of one query to the next in the array of queries. Thus order of the actions matters.



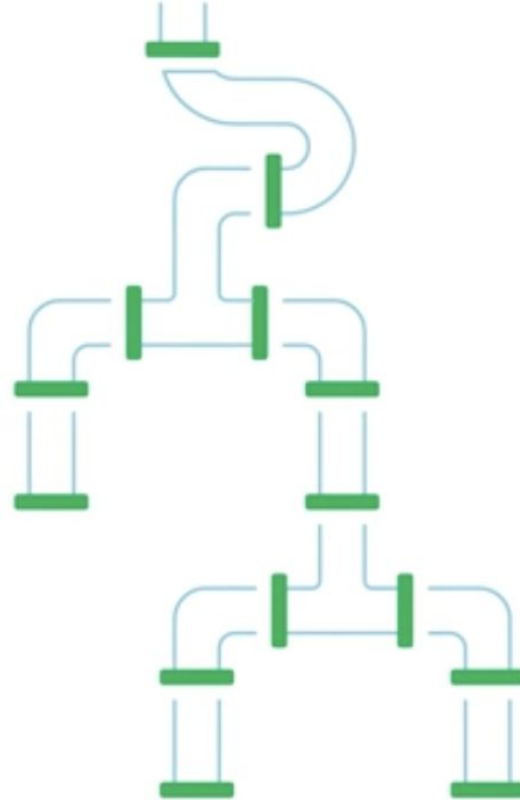
5. Aggregation Framework - Aggregation Pipeline

Pipeline diagram for the example Aggregation query

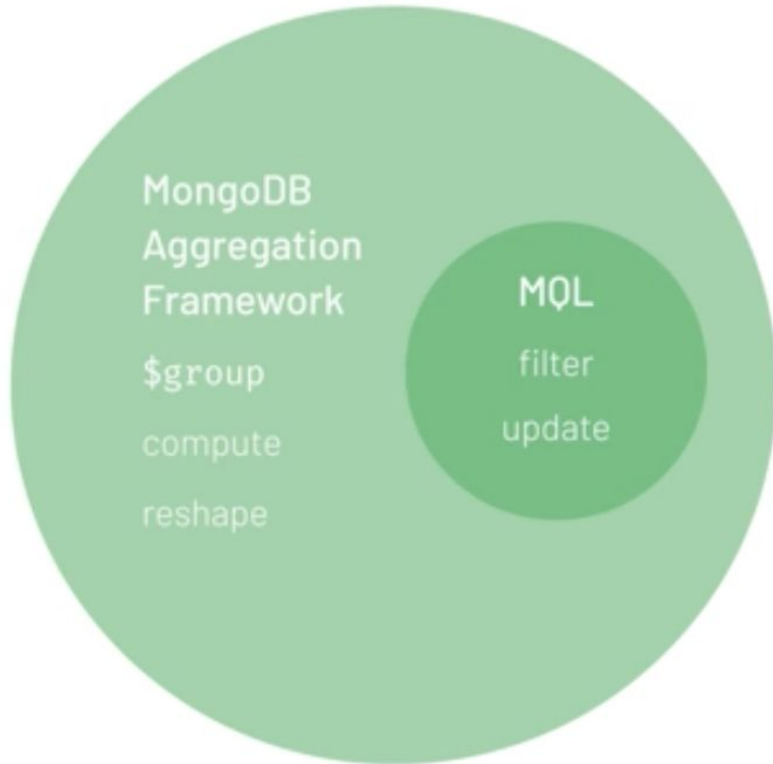


5. Aggregation Framework - Why Aggregation?

Why Aggregation?

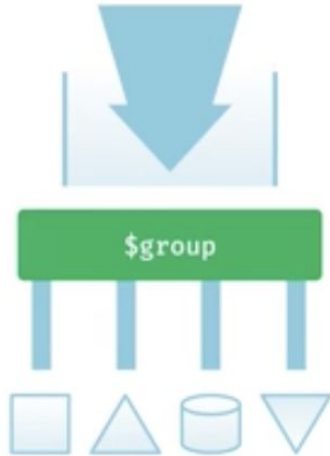


5. Aggregation Framework - Why Aggregation?



- With MQL we can filter & update data
- With Aggregation framework we can compute and reshape data.

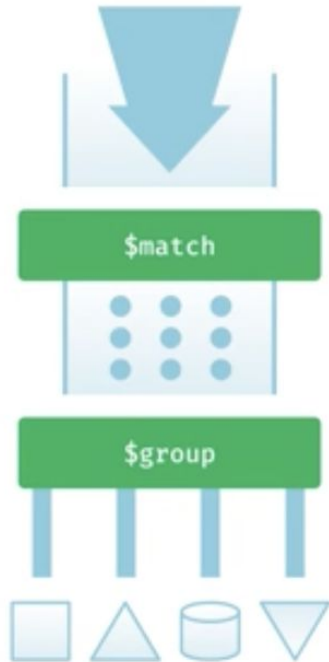
5. Aggregation Framework - \$group



`$group`

An operator that takes the incoming stream of data, and siphons it into multiple distinct reservoirs.

5. Aggregation Framework - \$group



Non- filtering stages do not modify the original data. Instead they work with the data in the cursor/

5. Aggregation Framework - \$group: Syntax

Syntax

```
{ $group:  
  {  
    _id: <expression>, // Group By Expression  
    <field1>: { <accumulator1> : <expression1> },  
    ... } }
```

5. Aggregation Framework - \$group: Example

\$group

How many distinct countries have listings, and how many listings does each country have?

```
db.listingsAndReviews.aggregate([  
    {$project: {"address":1,"_id":0}},  
    {$group: { _id:"$address.country",  
               "count":{"$sum":1}}}  
])
```

5. Aggregation Framework - \$group + \$sum

\$group + \$sum

```
{
  "$group":
  {
    "_id": "category",
    "total":
    { "$sum": "$price" }
  }
}
```

```
{
  { ... ,
    "category": "fish",
    "price": 5 },
  { ... ,
    "category": "meat",
    "price": 25 },
  { ... ,
    "category": "fish",
    "price": 7 }
}
```

```
{
  { "_id": "fish",
    "total": 12 },
  { "_id": "meat",
    "total": 25 }
}
```


5. Aggregation Framework - summary

Aggregation framework



Agg Framework > MQL



Pipeline stages in order



Syntax

5. Aggregation Framework - Examples

Example 1:

```
db.listingsAndReviews.aggregate([
  { "$match": { "amenities": "Wifi" } },
  { "$project": { "price": 1,
                  "address": 1,
                  "_id": 0 } }]).pretty()
```

Using the aggregation framework find all documents that have Wifi as one of the amenities`. Only include ``price and address in the resulting cursor.*

5. Aggregation Framework - Examples

Example 2:

```
db.listingsAndReviews.aggregate([ { "$project": { "address": 1, "_id": 0 }},  
                                  { "$group": { "_id": "$address.country" }}}])
```

*Project only the **address** field value for each document, then group all documents into one document per **address.country** value.*

5. Aggregation Framework - Examples

Example 3:

```
db.listingsAndReviews.aggregate([
  { "$project": { "address": 1, "_id": 0 } },
  { "$group": { "_id": "$address.country",
    "count": { "$sum": 1 } } }
])
```

*Project only the **address** field value for each document, then group all documents into one document per **address.country** value, and count one for each document in each group.*

5. Aggregation Framework - Quiz 1

Problem:

Write a query that finds all the room types that are present in the `sample_airbnb.listingsAndReviews` collection?

5. Aggregation Framework - Quiz 1

Problem:

Write a query that finds all the room types that are present in the `sample_airbnb.listingsAndReviews` collection?

```
db.listingsAndReviews.aggregate([ { "$group": { "_id": "$room_type" } }])
```

5. Aggregation Framework - Quiz 2

Problem:

What are the differences between using `aggregate()` and `find()`?

`aggregate()` allows us to compute and reshape data in the cursor.

`find()` can do what the `aggregate()` can do and more.

`aggregate()` can do what `find()` can and more.

`find()` allows us to compute and reshape data in the cursor.

5. Aggregation Framework - Quiz 2

Problem:

What are the differences between using `aggregate()` and `find()`?

<code>aggregate()</code> allows us to compute and reshape data in the cursor.	The aggregation framework allows us to compute and reshape data via using stages like <code>\$group</code> , <code>\$sum</code> , and others.
<code>find()</code> can do what the <code>aggregate()</code> can do and more.	Any <code>find()</code> query can be translated into an aggregation pipeline equivalent, but not every aggregation pipeline can be translated into a <code>find()</code> query.
<code>aggregate()</code> can do what <code>find()</code> can and more.	Any <code>find()</code> query can be translated into an aggregation pipeline equivalent, but not every aggregation pipeline can be translated into a <code>find()</code> query.
<code>find()</code> allows us to compute and reshape data in the cursor.	This is incorrect, the only way for us to compute and reshape data using <code>find()</code> , is by using the aggregation framework stages within the <code>find()</code> method.

5. sort() & limit()

sort() & limit()

5. sort() & limit() - cursor methods



Syntax

Cursor methods

`sort()`

`limit()`

`pretty()`

`count()`

5. sort() & limit() - cursor methods



find()
↓



Cursor

```
cursor.sort()
```

```
cursor.limit()
```

```
cursor.pretty()
```

```
cursor.count()
```

5. sort() & limit() - sort method

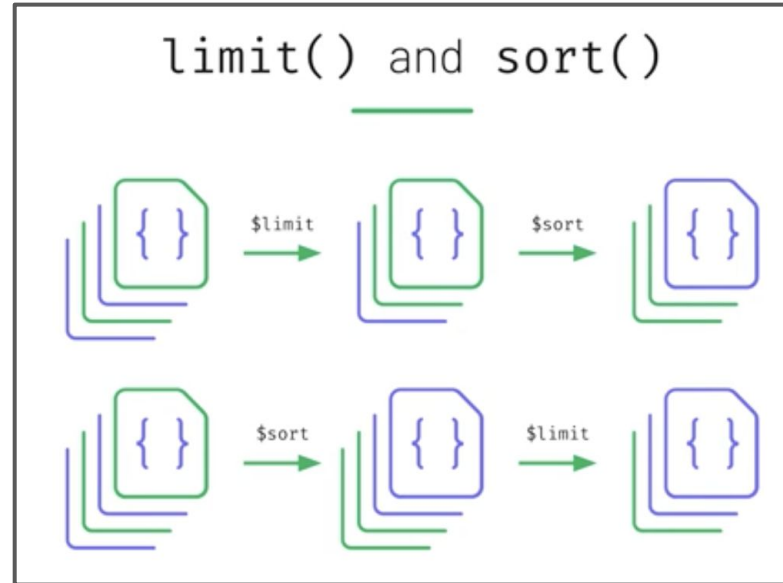
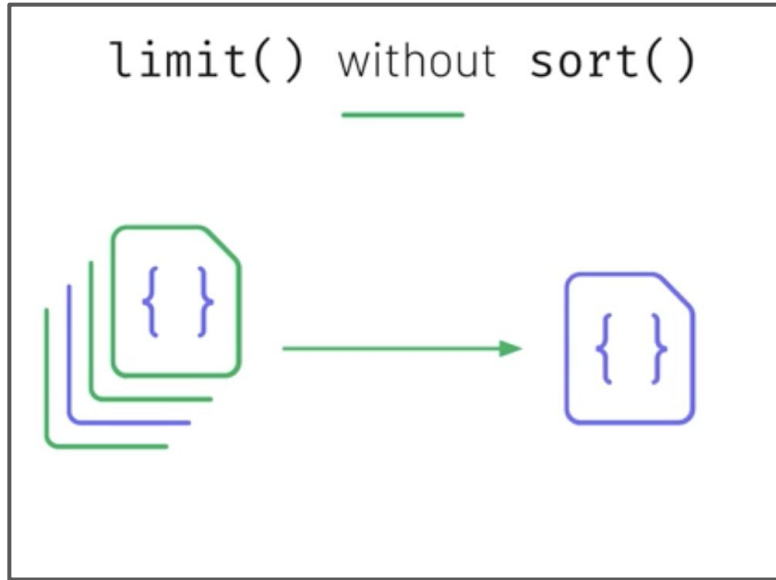
sort()

```
db.zips.find().sort({ "pop": 1, "city": -1 })
```

Increasing
"pop": 0 -> ∞

Decreasing
"city": "Z" -> "A"

5. sort() & limit() - limit method



5. sort() & limit() - order doesn't matter

```
cursor.limit().sort()
```

means

```
cursor.sort().limit()
```

5. sort() & limit() - Examples

Example 1:

```
db.zips.find().sort({ "pop": 1 }).limit(1)
```

```
db.zips.find().sort({ "pop": -1 }).limit(1)
```

```
db.zips.find().sort({ "pop": -1 }).limit(10)
```

```
db.zips.find().sort({ "pop": 1, "city": -1 })
```

5. sort() & limit() - Quiz 1

Problem:

Which of the following commands will return the name and founding year for the 5 oldest companies in the **companies** collection?

```
db.companies.find({ "founded_year": { "$ne": null }},  
  { "name": 1, "founded_year": 1 }  
).sort({ "founded_year": 1 }).limit(5)
```

```
db.companies.find({ "founded_year": { "$ne": null }},  
  { "name": 1, "founded_year": 1 }  
).limit(5).sort({ "founded_year": 1 })
```

```
db.companies.find({}, { "name": 1, "founded_year": 1 }  
).sort({ "founded_year": 1 }).limit(5)
```

```
db.companies.find({ "name": 1, "founded_year": 1 }  
).sort({ "founded_year": 1 }).limit(5)
```


5. sort() & limit() - Quiz 1

Problem:

Which of the following commands will return the name and founding year for the 5 oldest companies in the **companies** collection?

<pre>db.companies.find({ "founded_year": { "\$ne": null } }, { "name": 1, "founded_year": 1 }).sort({ "founded_year": 1 }).limit(5)</pre>	First filter out the documents where the founded year is not null, then project the fields that we are looking for, which is name, and founded_year. Then sort the cursor in increasing order, so the first results will have the smallest value for the founded_year field. Finally, limit the results to our top 5 documents in the cursor, thus getting the 5 oldest companies in this collection.
<pre>db.companies.find({ "founded_year": { "\$ne": null } }, { "name": 1, "founded_year": 1 }).limit(5).sort({ "founded_year": 1 })</pre>	While the limit() and sort() methods are not listed in the correct order, MongoDB flips their order when executing the query, delivering the results that the question prompt is looking for.
<pre>db.companies.find({ }, { "name": 1, "founded_year": 1 }).sort({ "founded_year": 1 }).limit(5)</pre>	This command is missing the search criteria, which means that null values are included. Excluding the null values isn't always necessary, because it isn't best practice to store these types of values in MongoDB anyway. So there are lots of collections out there without null values. However, this collection contains nulls, and we need to exclude them from our cursor, or else they'll be our smallest value when we sort.
<pre>db.companies.find({ "name": 1, "founded_year": 1 }).sort({ "founded_year": 1 }).limit(5)</pre>	It looks like the query filter has the values that the projection part of the find() command should have, and there is no projection used in this command. So this command will return all companies where the name is equal to 1 and the founded_year is also equal to 1, which unfortunately misses the requirements of the question prompt.

5. sort() & limit() - Quiz 2

Problem:

Define a query that finds what year was the youngest bike rider from the **trips** collection born?

5. sort() & limit() - Quiz 2

Problem:

Define a query that finds what year was the youngest bike rider from the **trips** collection born?

```
db.trips.find({ "birth year": { "$ne":"" } },  
             { "birth year": 1 }).sort({ "birth year": -1 }).limit(1)
```

The most important part is to first filter out all empty string values, in order to get actual year values. You don't have to use projection here, but we added it so that it is easier to see the answer. The sort order should be in decreasing order because we're looking for the youngest person, therefore the highest year value. You don't have to add the `limit()` method to get this result, but we also added it to make it easier to immediately see the result.

5. Introduction to Indexes

Introduction to Indexes

5. Introduction to Indexes - Why Index?

Indexes



Make queries even more efficient



Are one of the most impactful ways to improve query performance

5. Introduction to Indexes - What is Index?

Index

In a book — an alphabetical list of names, subjects, etc., with references to the places where they occur, typically found at the end of a book.

5. Introduction to Indexes - What is Index?

Not using an index

Find all mentions of Toni Morrison.

Look through every page in a book

Using an index

Find all mentions of Toni Morrison.

Go to **M** in the index

Locate **Morrison**

Go to referenced page

5. Introduction to Indexes - What is Index?

Index

In a database — special data structure that stores a small portion of the collection's data set in an easy to traverse form.

5. Introduction to Indexes - When to Index?

When to index

Support your queries

Given

```
db.trips.find({"birth year": 1989})
```

```
db.trips.find({"start station id": 476}).sort("birth year": 1)
```

Index

```
db.trips.createIndex({"birth year": 1})
```

5. Introduction to Indexes - How to Index

How is it better?

Index

```
db.trips.createIndex({"birth year": 1})
```

Queries



```
db.trips.find({"birth year": 1989})
```



```
db.trips.find({"start station id": 476}).sort("birth year": 1)
```

{station id: 476} —————> Use "birth year" index

5. Introduction to Indexes - Index vs Sort

Avoid sorting!

5. Introduction to Indexes - Compound Index

```
{"start station id": 1, "birth year": 1}
```



"station id": 12



"station id": 132



"station id": 390



"station id": 47

5. Introduction to Indexes - Compound Index

Can we do better?

Single field index

```
db.trips.createIndex({"birth year": 1})
```

Not perfect for

```
db.trips.find({"start station id": 476}).sort("birth year": 1)
```

Compound Index

```
db.trips.createIndex({"start station id": 1,"birth year": 1})
```

5. Introduction to Indexes - Compound Index

`{"start station id": 1, "birth year": 1}`



"station id": 12



"station id": 132



"station id": 390



"station id": 476



"station id": 12, 12,
"birth year": 1888 → 2010



"station id": 132, 132,
"birth year": 1888 → 2010



"station id": 390, 390,
"birth year": 1888 → 2010

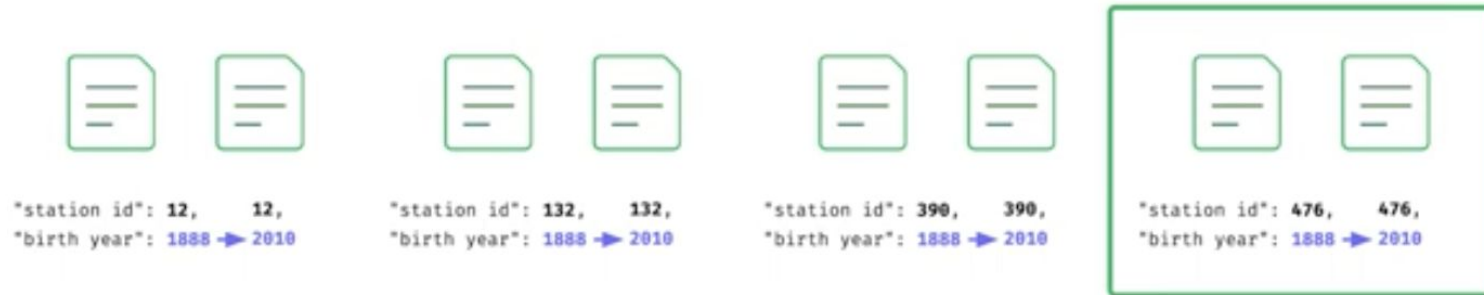


"station id": 476, 476,
"birth year": 1888 → 2010

```
db.trips.find({"start station id": 476}).sort("birth year": 1)
```

5. Introduction to Indexes - Compound Index

Compound Index

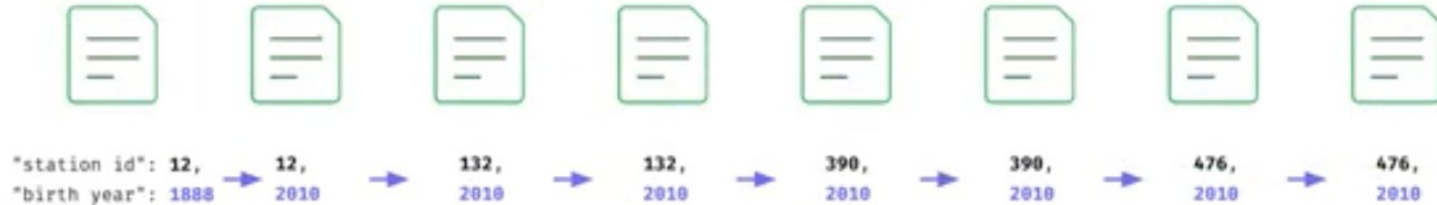


```
db.trips.find({"start station id": 476}).sort("birth year": 1)
```

5. Introduction to Indexes - Compound Index

Compound Index

"start station id": 476



```
db.trips.find({"start station id": 476}).sort("birth year": 1)
```


5. Introduction to Indexes - When to Index

When to index

Support your queries

Given

```
db.trips.find({"birth year": 1989})
```

```
db.trips.find({"start station id": 476}).sort("birth year
```

Index

```
db.trips.createIndex({"birth year
```

5. Introduction to Indexes - Examples

Example 1: (simple index)

```
db.trips.createIndex({ "birth year": 1 })
```

Example 2: (compound index)

```
db.trips.createIndex({ "start station id": 1, "birth year": 1 })
```

5. Introduction to Indexes - Quiz 1

Problem:

Jameela often queries the **routes** collection by the **src_airport** field like this:

```
db.routes.find({ "src_airport": "MUC" }).pretty()
```

Which command will create an index that will support this query?

```
db.routes.makeIndex({ "src_airport": 1 })
```

```
db.routes.createIndex({ "src_airport": -1 })
```

```
db.routes.addIndex({ "src_airport": 1 })
```

```
db.routes.generateIndex({ "src_airport": -1 })
```

5. Introduction to Indexes - Quiz 1

Problem:

Jameela often queries the **routes** collection by the **src_airport** field like this:

```
db.routes.find({ "src_airport": "MUC" }).pretty()
```

Which command will create an index that will support this query?

<code>db.routes.makeIndex({ "src_airport": 1 })</code>	<code>makeIndex()</code> is not a valid MongoDB command.
<code>db.routes.createIndex({ "src_airport": -1 })</code>	It doesn't really matter whether the index was created in increasing or decreasing order when it is a simple single-field index.
<code>db.routes.addIndex({ "src_airport": 1 })</code>	<code>addIndex()</code> is not a valid MongoDB command.
<code>db.routes.generateIndex({ "src_airport": -1 })</code>	<code>db.routes.generateIndex({ "src_airport": -1 })</code>

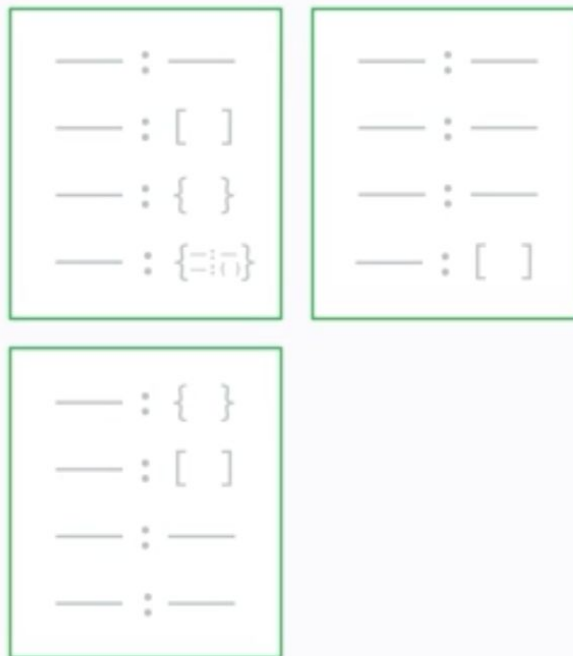
5. Introduction to Data Modeling

Introduction to Data Modeling

5. Introduction to Data Modeling - What is Data Modeling?

What is data modeling?

a way to organize fields in a document to support your application performance and querying capabilities.



5. Introduction to Data Modeling - General Rule

Rule:

Data is stored in the
way that it is used.

5. Introduction to Data Modeling - Example



Patient 1

```
{ "preferred_cont":  
  {"type": "email",  
   "cont": "a@a1.com" },  
  "contact_info": [  
    {"type": "work #",  
     "cont": "11111111"},  
    {"type": "cell #",  
     "cont": "22222222"}],  
  "prescriptions" :  
    [234,039, 50],  
  "visit_hist": [  
    {"date": "2020-12-10",  
     "notes": "..."}]  
}
```


5. Introduction to Data Modeling - Example



Patient 2

```
{ "preferred_cont":  
  {"type": "email",  
    "cont": "b@bc.com" },  
  "contact_info": [  
    {"type": "cell #",  
      "cont": "77777777"}]  
}
```

5. Introduction to Data Modeling - Example



Main considerations

What we'll store

Contact information

Visit history

Prescriptions

Age

Gender

Etc.

How it will be queried

Who is using our application?

5. Introduction to Data Modeling - Example

What doctors need

Current prescriptions

Diagnoses

Patient contact information

Ability to cross-reference the medication information

- Side-effects
- Allergies
- Etc.

5. Introduction to Data Modeling - Example

Organize

```
{  
  "name": "", "age": #,  
  "pref cont": "", "conts": [{},{},{}],  
  "prescriptions": [{},{},{}],  
  "allergies": [], "prior visits": [{}],  
  "next visit": "", "diagnoses": []  
}
```

```
{  
  "name": "", "code": #,  
  "side effects": [], "allergens": [],  
  "active ingredients": [],  
  "interaction with other drugs": [{}],  
  "uses": []  
}
```

Query

```
db.patient.find({"name": "Cora"})
```

```
db.patient.find({"next visit": "12-15"})
```

```
db.medication.find({"uses": "flu"})
```

```
db.medication.find({"code": 329})
```



5. Introduction to Data Modeling - MongoDB

Data modeling with MongoDB

Data that is **used** together should be **stored** together

Evolving application implies an evolving data model

5. Introduction to Data Modeling - Quiz 1

Problem:

What is data modeling? Choose the best answer.

a way to organize fields in a document to support your application performance and querying capabilities

a way to show off your amazing data to everyone using graphs and illustrations

a way to build your application based on how your data is stored

a way to decide whether to store your data in the cloud or locally

5. Introduction to Data Modeling - Quiz 1

Problem:

What is data modeling? Choose the best answer.

a way to organize fields in a document to support your application performance and querying capabilities	Data modeling is a way to organize your data, which includes making decisions about fields, collections, and datatypes that will be used in each collection.
a way to show off your amazing data to everyone using graphs and illustrations	Showing off data using graphs and illustrations is called data visualization and is not part of data modeling.
a way to build your application based on how your data is stored	You can build your application however you see fit. When using some databases you may have to build your application around how the data is stored, but that is not the case for MongoDB. Here, you use data modeling to make decisions that work best with your application and not the other way around.
a way to decide whether to store your data in the cloud or locally	The decision about what type of storage you are going to use pertains more to the area of security and systems engineering and does not relate to data modeling.

5. Upsert - Update or Insert?

Upsert -
Update or Insert?

5. Upsert - Update or Insert?

Upsert

Everything in MQL that is used to locate a document in a collection can also be used to **modify** this document.

```
db.collection.updateOne({<query to locate>},{<update>})
```

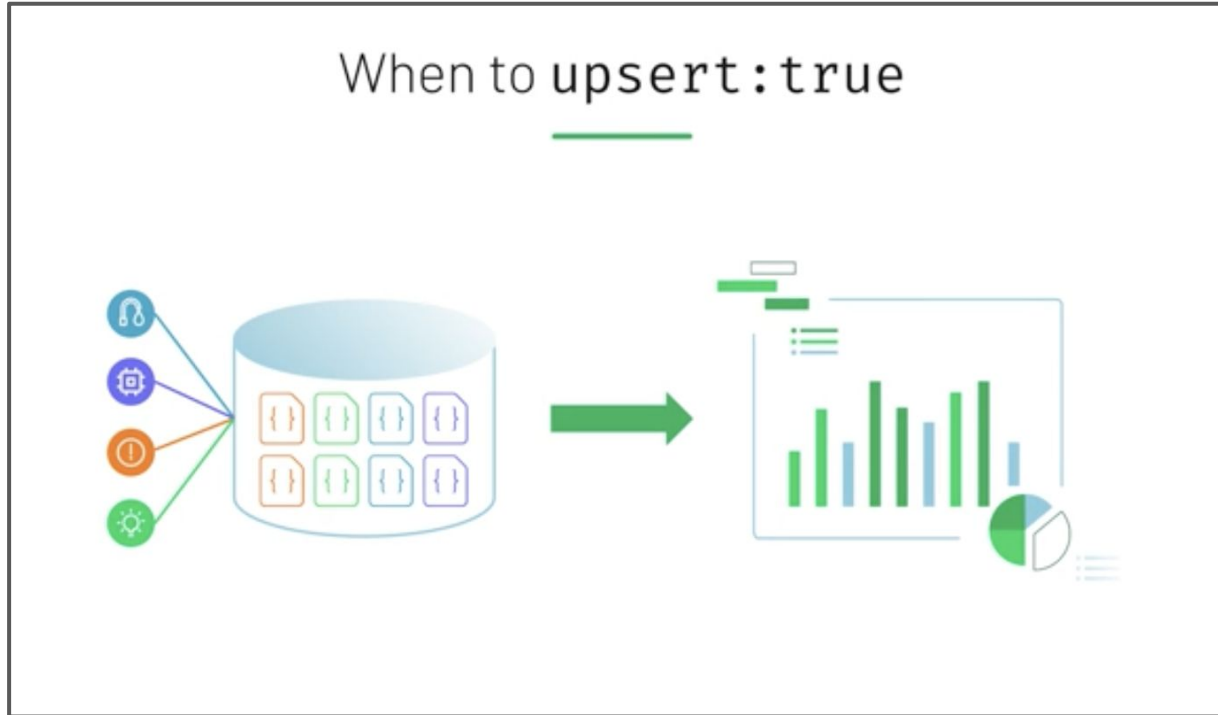
Upsert is a hybrid of update and insert, it should only be used when it is needed.

```
db.collection.updateOne({<query>},{<update>},{"upsert":true})
```

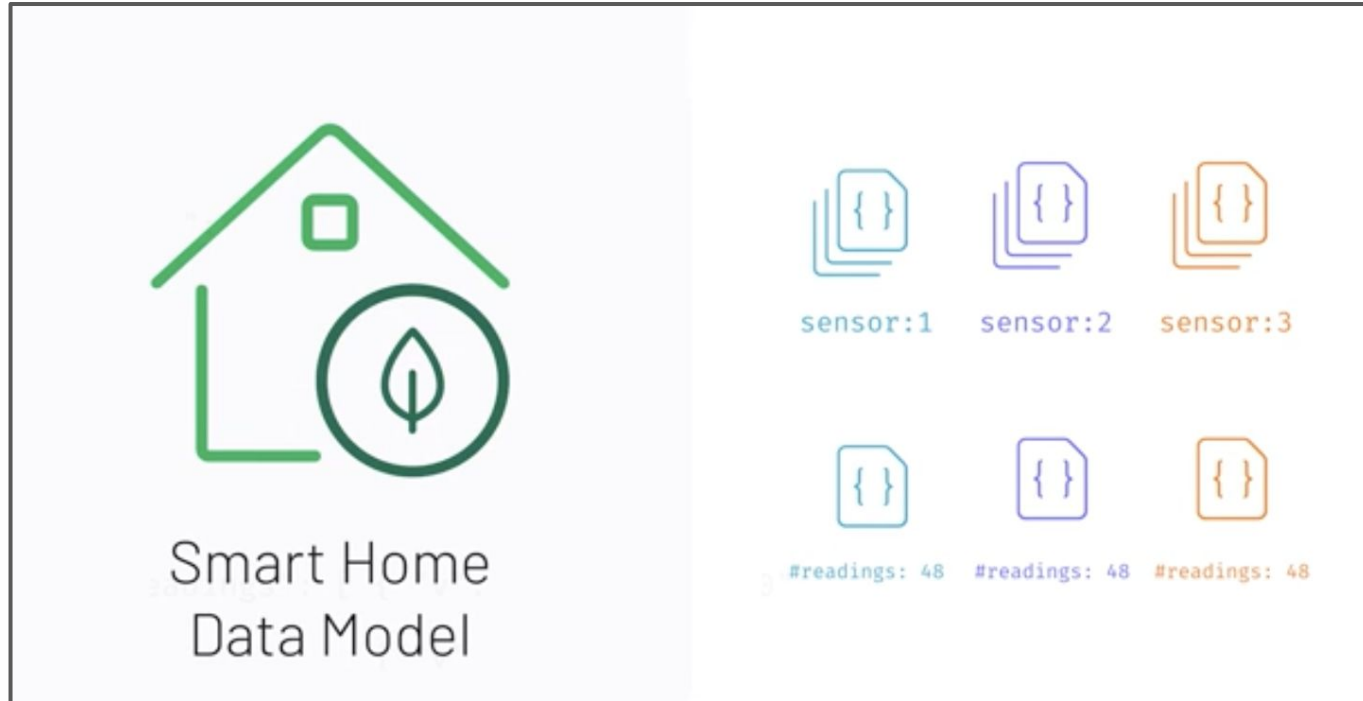
5. Upsert - Update or Insert?



5. Upsert - Update or Insert? - Example



5. Upsert - Update or Insert? - Example



5. Upsert - Update or Insert? - Example

Why upsert:true

```
{ "_id": ObjectId("abcd12340101"),  
  "sensor": 5,  
  "date": Date("2021-05-11"),  
  "valcount": 2,  
  "tot": 144,  
  "readings": [ { "v": 70, "t": "0000"},  
                 { "v": 74, "t": "0005"} ] }
```

5. Upsert - Update or Insert? - Example

Why `upsert:true`

New reading `r!`

`sensor = 5,`

`value = 72,`

`date = Date('2021-05-11'),`

`time = "0010"`

5. Upsert - Update or Insert? - Example

How to upsert

```
db.iot.updateOne({
  "sensor": r.sensor,
  "date": r.date,
  "valcount": { "$lt": 48 }
},
{ "$push": { "readings":
  { "v": r.value,
    "t": r.time }
},
  "$inc": { "valcount": 1,
    "total": r.value }},
{ upsert: true })
```

Current document

```
{
  "_id": ObjectId("abcd12345309"),
  "sensor": 5,
  "date": Date("2021-05-11"),
  "valcount": 1,
  "tot": 72
}
```

5. Upsert - Update or Insert? - Example

How to upsert

```
db.iot.updateOne({
  "sensor": r.sensor,
  "date": r.date,
  "valcount": { "$lt": 48 }
},
{ "$push": { "readings":
  { "v": r.value,
    "t": r.time }
  },
  "$inc": { "valcount": 1,
    "total": r.value }},
{ upsert: true })
```

Current document

```
{
  "_id": ObjectId("abcd12345309"),
  "sensor": 5,
  "date": Date("2021-05-11"),
  "valcount": 1,
  "tot": 72,
  "readings": [{ "v": 72,
    "t": "0010"},
  ]
}
```


5. Upsert - Update or Insert? - Considerations

`upsert:true`

Be mindful

Is {<update>} enough to create a new document?

Will the document have the same or similar form to other documents in the collection?

5. Upsert - Update or Insert? - Summary

Summary

`upsert : true`

Conditional updates

`upsert : false`

Update an existing document

Insert a brand new document

5. Upsert - Update or Insert? - Quiz 1

Problem:

How does the upsert option work? Check all answers that apply:

When upsert is set to true and the query predicate returns an empty cursor, the update operation creates a new document using the directive from the query predicate and the update predicate.

When upsert is set to false and the query predicate returns an empty cursor then there will be no updated documents as a result of this operation.

By default upsert is set to false

It is used with the update operator, and needs to have its value specified every time that the update operator is called.

5. Upsert - Update or Insert? - Quiz 1

Problem:

How does the upsert option work? Check all answers that apply:

When upsert is set to true and the query predicate returns an empty cursor, the update operation creates a new document using the directive from the query predicate and the update predicate.	When upsert is set to true it can perform an insert if the query predicate doesn't return a matching document.
When upsert is set to false and the query predicate returns an empty cursor then there will be no updated documents as a result of this operation.	When upsert is set to false an update will happen only when the query predicate is matched with a document from the collection.
By default upsert is set to false	If the upsert option is not specified, then it will have the value of false by default.
It is used with the update operator, and needs to have its value specified every time that the update operator is called.	The upsert option only needs its value specified if you want to change the default false setting to true.

6. Next Steps

Part 6

Next Steps

6. Next Steps

- **Atlas Features - More Data Explorer**
- **Atlas Products & Options**
- **What is MongoDB Compass?**

6. Atlas Features - More Data Explorer

Atlas Features - More Data Explorer

6. Atlas Features - More Data Explorer - Quiz 1

Problem:

What actions are available to you via the Aggregation Builder in the Atlas Data Explorer? Select all that apply.

Export pipeline to a programming language.

A preview of the data in the pipeline at each selected stage.

An indication of which indexes are used by the pipeline.

Syntax for each selected aggregation stage.

6. Atlas Features - More Data Explorer - Quiz 1

Problem:

What actions are available to you via the Aggregation Builder in the Atlas Data Explorer? Select all that apply.

Export pipeline to a programming language.	You can choose to export the pipeline to one of four programming languages, and select to view the import statements that are needed in the application for this pipeline to work.
A preview of the data in the pipeline at each selected stage.	There is a sample of data, or the whole result set to the right of every stage that uses correct syntax in the aggregation pipeline builder in Atlas.
An indication of which indexes are used by the pipeline.	There is no information pertaining to indexes in the Atlas Aggregation Builder.
Syntax for each selected aggregation stage.	When you select a stage the space to enter stage instructions has a syntax outline to help you recall what information the stage requires and in what format.

6. Atlas Products & Options

Atlas Products & Options

6. Atlas Products & Options - Quiz 1

Problem:

What is MongoDB Charts? Select all that apply.

A feature that displays performance data about your Atlas Cluster.

A product that helps you build visualizations of the data stored in your Atlas Cluster.

An app that allows you to embed visualizations that are created in other applications into your website.

6. Atlas Products & Options - Quiz 1

Problem:

What is MongoDB Charts? Select all that apply.

A feature that displays performance data about your Atlas Cluster.	Atlas has a Performance advisor with regards to the index use, and schema anti-patterns. You can also see cluster performance metrics in Atlas, but that is not part of the Charts product.
A product that helps you build visualizations of the data stored in your Atlas Cluster.	Charts is available for all MongoDB Atlas Clusters and allows you to build visualizations of your data using the aggregation pipeline or even the MQL to filter data before creating charts.
An app that allows you to embed visualizations that are created in other applications into your website.	Charts allows you to embed dashboards into your application or website, but those dashboards are created in Charts, not in other applications.

6. What is MongoDB Compass?

What is
MongoDB Compass?

6. What is MongoDB Compass? - Quiz 1

Problem:

What is MongoDB Compass? Select all that apply.

A term used to denote an object that stores geospatial coordinates in the collection.

MongoDB's Graphical User Interface Product

A type of map-based chart that is available with MongoDB Charts

6. What is MongoDB Compass? - Quiz 1

Problem:

What is MongoDB Compass? Select all that apply.

A term used to denote an object that stores geospatial coordinates in the collection.	There is no such term in MongoDB. Coordinates are stored as numeric BSON types.
MongoDB's Graphical User Interface Product	Compass is a Graphical User Interface that provides a lot of support in querying, visualizing, and analyzing your data.
A type of map-based chart that is available with MongoDB Charts	There isn't a map-based chart in MongoDB charts that is called Compass.

End

End

Go get Mongo Certified:

<https://university.mongodb.com/courses/M001/about>